

Capstone Project

Project Title: Customer Segmentation for Strategic Sales Management in the Food Wholesale Sector

Step 1: Problem Understanding & Framing

Business Problem:

A food wholesale distributor serves customers who purchase different amounts across several product categories, including Fresh, Milk, Grocery, Frozen, Detergents and Paper, and Delicatessen products. However, looking only at individual sales figures may not clearly show the different types of customers or their overall purchasing behaviour.

The business needs to understand:

- Which product categories receive the highest annual spending
- Whether customers show similar purchasing patterns
- Whether purchasing behaviour differs between sales channels
- Whether customer region is associated with spending patterns
- Which customer groups may benefit from different sales, marketing, or account-management strategies

Without clear customer segmentation, the distributor may apply the same strategy to all customers even though their purchasing needs and value to the business may be different.

Data Science Problem:

The data science problem is to use unsupervised machine learning to group wholesale customers according to similarities in their annual spending across different product categories.

Clustering algorithms will identify naturally occurring customer segments without using a predefined target variable. The resulting clusters will then be analysed to understand:

- The main spending characteristics of each cluster
- The product categories that distinguish the clusters

- The distribution of channels and regions across the clusters
- Possible sales and marketing strategies for each customer segment

Task Type:

This project will primarily be a clustering task using unsupervised machine learning.

The project will compare clustering methods such as:

- K-Means
- Hierarchical Clustering
- DBSCAN

Exploratory data analysis will also be used to examine product spending, channel differences, regional differences, correlations, and outliers. However, Exploratory Data Analysis (EDA) supports the clustering task and is not a separate machine-learning task.

Project questions:

The project aims to answer the following questions:

1. Which product categories account for the highest annual customer spending?
2. Are there distinct groups of customers with similar purchasing patterns?
3. Which product categories are most important in separating the customer groups?
4. Do customer segments differ according to sales channel?
5. Do customer segments differ according to region?
6. How can the identified segments support sales and account-management strategies?

Success metrics

The clustering models will be evaluated using:

- **Silhouette Score:** Measures how well each customer fits within its assigned cluster compared with other clusters. A higher score indicates better-defined clusters.

- **Davies–Bouldin Index:** Measures how similar the clusters are to each other. A lower score indicates better separation between clusters.
- **Calinski–Harabasz Score:** Measures how compact and well-separated the clusters are. A higher score indicates stronger clustering performance.
- **Cluster stability:** Assesses whether similar clusters are produced when the model settings, random seed, or sample are changed.
- **Business interpretability:** Determines whether the clusters have clear and meaningful purchasing profiles that can support practical business decisions.
- **Cluster balance:** Checks whether the model produces useful cluster sizes rather than placing most customers into one cluster or creating several extremely small groups.

Business KPIs

The business value of the project will be assessed through:

- Identifying clear customer segments based on annual purchasing behaviour.
- Understanding which product categories are most important within each customer segment.
- Determining whether purchasing patterns differ between sales channels.
- Determining whether purchasing patterns differ across regions.
- Identifying high-spending, specialised, low-spending, or potentially valuable customer groups.
- Supporting more targeted sales, marketing, and account-management strategies.
- Identifying possible cross-selling opportunities between product categories.
- Improving how sales and marketing resources are allocated across different customer groups.
- Producing customer segments that are simple enough for business teams to understand and use.

Step 2: Data Collection and Understanding

Data Source and Justification

The Wholesale Customers dataset was obtained from the UCI Machine Learning Repository. It contains information about customers of a wholesale distributor and records their annual spending across six product categories. The dataset contains 440 customer records and is publicly available under a Creative Commons Attribution 4.0 licence, meaning it can be used and shared in the project as long as the original source is acknowledged.

This dataset was selected because:

- It is publicly available and does not contain confidential company information.
- It contains real-world customer spending data that is relevant to sales and account management.
- It is small enough to analyse within the available project timeframe.
- It contains enough variation in customer spending to support meaningful clustering.
- It allows customer groups to be compared by sales channel and region.
- UCI identifies clustering as one of the suitable machine-learning tasks for the dataset.

The data reference is as follows, Cardoso, M. (2013). *Wholesale customers* [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5030X.

Dataset Overview

Inspection of the uploaded CSV file showed that:

- The dataset contains 440 rows, with each row representing one wholesale customer.
- It contains 8 columns.
- Six columns represent annual customer spending:
 - Fresh
 - Milk
 - Grocery
 - Frozen
 - Detergents and Paper
 - Delicatessen

- Two columns describe the customer:
 - Channel
 - Region
- All columns are stored as integers.
- Channel and Region are coded as integers but are conceptually categorical variables.
- The six spending variables are numerical variables measured in unspecified monetary units.
- The dataset does not contain a customer identification column.
- There is no target variable because this is an unsupervised clustering project.
- There are no missing values.
- There are no duplicated rows.

The dataset records annual spending rather than the quantity of products sold. Therefore, the analysis can identify which categories receive the highest customer spending, but it cannot determine the number or physical volume of products sold.

Feature Types

The variables can be separated into two groups.

Numerical spending variables

- Fresh
- Milk
- Grocery
- Frozen
- Detergents_Paper
- Delicassen

These variables will initially be used to create the customer clusters.

Categorical customer variables

- Channel
- Region

Channel and Region Distribution

The dataset documentation identifies the Channel categories as **Horeca** and **Retail**, and the Region categories as **Lisbon**, **Oporto**, and **Other**. The CSV file only contains numeric codes, so the labels were matched using the category frequencies reported in the dataset documentation. Based on this matching, **Channel 1 represents Horeca, Channel 2 represents Retail, Region 1 represents Lisbon, Region 2 represents Oporto, and Region 3 represents Other regions**. However, because the official UCI page does not clearly show the numeric coding, this mapping should be noted as being derived from the published category frequencies rather than directly stated in the CSV.

Initial Distribution Findings

The six spending variables have positively skewed distributions. This means that most customers have low-to-moderate annual spending, while a smaller number of customers have extremely high spending.

The average annual spending by product category is:

- Fresh: 12,000.30 monetary units
- Grocery: 7,951.28 monetary units
- Milk: 5,796.27 monetary units
- Frozen: 3,071.93 monetary units
- Detergents and Paper: 2,881.49 monetary units
- Delicatessen: 1,524.87 monetary units

Fresh products have the highest average annual spending, followed by Grocery and Milk. However, averages may be influenced by the small number of customers with very high spending.

Missing Values and Duplicates

Initial data-quality checks found:

- No missing values in any column.
- No completely duplicated rows.
- No negative spending values.
- All Channel values are either 1 or 2.

- All Region values are 1, 2 or 3.

The dataset is therefore relatively clean, although additional preprocessing will still be necessary due to skewed distributions, differences in variable scale and extreme values.

Initial Outlier Assessment

Potential outliers were identified using the Interquartile Range method, where values below:

$$Q1 - 1.5 \times IQR$$

or above:

$$Q3 + 1.5 \times IQR$$

were treated as potential outliers.

The number of potential outliers identified in each spending category was:

- Fresh: 20
- Milk: 28
- Grocery: 24
- Frozen: 43
- Detergents_Paper: 30
- Delicassen: 27

These values should not automatically be deleted. In a customer-sales dataset, unusually high values may represent genuine high-value customers rather than data-entry errors.

Data Dictionary

Variable	Stored type	Analytical type	Description	Units or allowed values
Channel	Integer	Categorical	Customer's purchasing channel	1 = Horeca, 2 = Retail
Region	Integer	Categorical	Region where the customer operates	1 = Lisbon, 2 = Oporto, 3 = Other

Fresh	Integer	Numerical, continuous	Annual spending on fresh products	Monetary units
Milk	Integer	Numerical, continuous	Annual spending on milk products	Monetary units
Grocery	Integer	Numerical, continuous	Annual spending on grocery products	Monetary units
Frozen	Integer	Numerical, continuous	Annual spending on frozen products	Monetary units
Detergents_Paper	Integer	Numerical, continuous	Annual spending on detergent and paper products	Monetary units
Delicassen	Integer	Numerical, continuous	Annual spending on delicatessen products	Monetary units

Codes and Output

STEP 2: DATA COLLECTION AND UNDERSTANDING

```
import pandas as pd
```

```
import numpy as np
```

```
# -----
```

```
# 1. LOAD THE DATASET
```

```
# -----
```

```
file_path = "/content/Wholesale customers data.csv"
```

```
df = pd.read_csv(file_path)
```



```
print("Dataset loaded successfully.")
```

```
print()
```

```
# -----
```

```
# 2. DISPLAY THE FIRST FIVE ROWS
```

```
# -----
```

```
print("First five rows:")
```

```
display(df.head())
```

```
# -----
```

```
# 3. CHECK DATASET SIZE
```

```
# -----
```

```
print("Dataset shape:")
```

```
print(f"Rows: {df.shape[0]}")
```

```
print(f"Columns: {df.shape[1]}")
```

```
print()
```

```
# -----
```

```
# 4. DISPLAY COLUMN NAMES
```

```
# -----
```

```
print("Column names:")
```

```
print(df.columns.tolist())
```

```
print()
```

```
# -----
```

```
# 5. CHECK DATA TYPES
```

```
# -----
```

```
print("Data types:")
```

```
display(df.dtypes.to_frame(name="Data Type"))
```

```
# -----
```

```
# 6. GENERAL DATASET INFORMATION
```

```
# -----
```

```
print("Dataset information:")
```

```
df.info()
```

```
# -----
```

```
# 7. CHECK MISSING VALUES
```

```
# -----
```

```
missing_values = pd.DataFrame({  
    "Missing Values": df.isnull().sum(),  
    "Missing Percentage": (df.isnull().sum() / len(df) * 100).round(2)  
})
```

```
print("\nMissing-value summary:")
display(missing_values)
```

```
# -----
```

```
# 8. CHECK DUPLICATED ROWS
```

```
# -----
```

```
duplicate_count = df.duplicated().sum()
```

```
print(f"Number of duplicated rows: {duplicate_count}")
```

```
# -----
```

```
# 9. DESCRIPTIVE STATISTICS
```

```
# -----
```

```
print("\nDescriptive statistics:")
```

```
display(df.describe().T)
```

```
# -----
```

```
# 10. CHANNEL AND REGION COUNTS
```

```
# -----
```

```
channel_mapping = {
```

```
    1: "Horeca",
```

```
    2: "Retail"
```

```
}
```

```
region_mapping = {
```

```
    1: "Lisbon",
```

```
    2: "Oporto",
```

```
    3: "Other"
```

```
}
```

```
channel_counts = df["Channel"].value_counts().sort_index().rename(
```

```
    index=channel_mapping
```

```
)
```

```
region_counts = df["Region"].value_counts().sort_index().rename(
```

```
    index=region_mapping
```

```
)
```

```
print("\nChannel distribution:")
```

```
display(channel_counts.to_frame(name="Customer Count"))
```

```
print("\nRegion distribution:")
```

```
display(region_counts.to_frame(name="Customer Count"))
```

```
# -----
```

```
# 11. IDENTIFY SPENDING VARIABLES
```

```
# -----
```

```
spending_columns = [
```

```

    "Fresh",
    "Milk",
    "Grocery",
    "Frozen",
    "Detergents_Paper",
    "Delicassen"
]

# -----
# 12. SUMMARISE SPENDING VARIABLES
# -----

spending_summary = df[spending_columns].describe().T

spending_summary["median"] = df[spending_columns].median()
spending_summary["skewness"] = df[spending_columns].skew()

spending_summary = spending_summary[
    ["count", "mean", "median", "std", "min", "25%", "50%", "75%",
     "max", "skewness"]
]

print("\nSpending-variable summary:")
display(spending_summary.round(2))

# -----

```

```
# 13. CHECK FOR INVALID VALUES
```

```
# -----
```

```
negative_values = (df[spending_columns] < 0).sum()
```

```
print("\nNegative spending values:")
```

```
display(negative_values.to_frame(name="Negative Value Count"))
```

```
print("\nUnique Channel values:")
```

```
print(sorted(df["Channel"].unique()))
```

```
print("\nUnique Region values:")
```

```
print(sorted(df["Region"].unique()))
```

```
# -----
```

```
# 14. IDENTIFY POTENTIAL OUTLIERS USING IQR
```

```
# -----
```

```
outlier_summary = []
```

```
for column in spending_columns:
```

```
    q1 = df[column].quantile(0.25)
```

```
    q3 = df[column].quantile(0.75)
```

```
    iqr = q3 - q1
```

```
    lower_limit = q1 - (1.5 * iqr)
```

```
    upper_limit = q3 + (1.5 * iqr)
```

```
outlier_count = (  
    (df[column] < lower_limit) |  
    (df[column] > upper_limit)  
).sum()
```

```
outlier_summary.append({  
    "Variable": column,  
    "Q1": q1,  
    "Q3": q3,  
    "IQR": iqr,  
    "Lower Limit": lower_limit,  
    "Upper Limit": upper_limit,  
    "Potential Outliers": outlier_count  
})
```

```
outlier_summary_df = pd.DataFrame(outlier_summary)
```

```
print("\nPotential outliers using the IQR method:")
```

```
display(outlier_summary_df.round(2))
```

```
# -----
```

```
# 15. CREATE DATA DICTIONARY
```

```
# -----
```

```
data_dictionary = pd.DataFrame({  
    "Variable": [  

```

```
"Channel",
"Region",
"Fresh",
"Milk",
"Grocery",
"Frozen",
"Detergents_Paper",
"Delicassen"
],
"Analytical Type": [
"Categorical",
"Categorical",
"Numerical",
"Numerical",
"Numerical",
"Numerical",
"Numerical",
"Numerical",
"Numerical"
],
"Description": [
"Customer purchasing channel",
"Customer operating region",
"Annual spending on fresh products",
"Annual spending on milk products",
"Annual spending on grocery products",
"Annual spending on frozen products",
"Annual spending on detergents and paper products",
"Annual spending on delicatessen products"
```



```

],
"Units / Allowed Values": [
    "1 = Horeca, 2 = Retail",
    "1 = Lisbon, 2 = Oporto, 3 = Other",
    "Monetary units",
    "Monetary units",
    "Monetary units",
    "Monetary units",
    "Monetary units",
    "Monetary units"
]
})

```

```

print("\nData dictionary:")
display(data_dictionary)

```

Dataset loaded successfully.

First five rows:

	Chann el	Regio n	Fres h	Mil k	Grocer y	Froze n	Detergents_Pa per	delicatess en
0	2	3	1266 9	965 6	7561	214	2674	1338
1	2	3	7057	981 0	9568	1762	3293	1776
2	2	3	6353	880 8	7684	2405	3516	7844

	Chann el	Regio n	Fres h	Mil k	Grocer y	Froze n	Detergents_Pa per	delicatess en
3	1	3	1326 5	119 6	4221	6404	507	1788
4	2	3	2261 5	541 0	7198	3915	1777	5185

Dataset shape:

Rows: 440

Columns: 8

Column names:

['Channel', 'Region', 'Fresh', 'Milk', 'Grocery', 'Frozen', 'Detergents_Paper', 'delicatessen']

Data types:

	Data Type
Channel	int64
Region	int64
Fresh	int64
Milk	int64
Grocery	int64
Frozen	int64
Detergents_Paper	int64
delicatessen	int64

Dataset information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 440 entries, 0 to 439

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	Channel	440 non-null	int64
1	Region	440 non-null	int64
2	Fresh	440 non-null	int64
3	Milk	440 non-null	int64
4	Grocery	440 non-null	int64
5	Frozen	440 non-null	int64
6	Detergents_Paper	440 non-null	int64
7	delicatessen	440 non-null	int64

dtypes: int64(8)

memory usage: 27.6 KB

Missing-value summary:

	Missing Values	Missing Percentage
Channel	0	0.0
Region	0	0.0
Fresh	0	0.0
Milk	0	0.0
Grocery	0	0.0

	Missing Values	Missing Percentage
Frozen	0	0.0
Detergents_Paper	0	0.0
delicatessen	0	0.0

Number of duplicated rows: 0

Descriptive statistics:

	count	mean	std	min	25%	50%	75%	max
Channel	440.0	1.322727	0.468052	1.0	1.00	1.0	2.00	2.0
Region	440.0	2.543182	0.774272	1.0	2.00	3.0	3.00	3.0
Fresh	440.0	12000.297727	12647.328865	3.0	3127.75	8504.0	16933.75	112151.0
Milk	440.0	5796.265909	7380.377175	55.0	1533.00	3627.0	7190.25	73498.0
Grocery	440.0	7951.277273	9503.162829	3.0	2153.00	4755.5	10655.75	92780.0
Frozen	440.0	3071.931818	4854.673333	25.0	742.25	1526.0	3554.25	60869.0
Detergents_Paper	440.0	2881.493182	4767.854448	3.0	256.75	816.5	3922.00	40827.0

	count	mean	std	min	25%	50%	75%	max
delicatessen	440.	1524.870	2820.105	3.0	408.2	965.	1820.	47943
	0	455	937		5	5	25	.0

Channel distribution:

Customer Count	
Channel	
Horeca	298
Retail	142

Region distribution:

Customer Count	
Region	
Lisbon	77
Oporto	47
Other	316

Step 3: Data Preprocessing, Applied EDA & Feature Engineering

This section prepares the dataset for clustering and examines the main patterns within the customer spending data. The analysis is divided into four stages: data cleaning and validation, exploratory data analysis, feature engineering and feature selection, and data transformation with dimensionality reduction. These steps ensure that the final clustering models are based on clean, relevant and appropriately scaled data.

Data Cleaning and Validation

The dataset was checked for missing values, duplicate rows, invalid category codes and incorrect spending values. It contained 440 rows and 8 columns, with no missing values, duplicate records, negative values or zero spending values.

Channel contained the valid codes 1 and 2, while Region contained 1, 2 and 3. These variables were converted to categorical data types because they represent groups rather than numerical measurements.

No records were removed. Unusually high spending values were kept because they may represent genuine high-value customers and will be examined during exploratory data analysis.

Code and Output:

```
import pandas as pd
```

```
df = pd.read_csv("/content/Wholesale customers data.csv")
```

```
df_clean = df.copy()
```

```
spending_columns = [
```

```
    "Fresh",
```

```
    "Milk",
```

```
    "Grocery",
```

```
    "Frozen",
```

```
    "Detergents_Paper",
```

```
    "Delicassen"
```

```
]
```

```
print("Shape:", df_clean.shape)
```

```
print("\nMissing values:")
```

```
print(df_clean.isnull().sum())
```

```
print("\nDuplicate rows:")  
print(df_clean.duplicated().sum())
```

```
print("\nNegative values:")  
print((df_clean[spending_columns] < 0).sum())
```

```
print("\nZero values:")  
print((df_clean[spending_columns] == 0).sum())
```

```
print("\nChannel codes:")  
print(sorted(df_clean["Channel"].unique()))
```

```
print("\nRegion codes:")  
print(sorted(df_clean["Region"].unique()))
```

```
df_clean["Channel"] = df_clean["Channel"].astype("category")  
df_clean["Region"] = df_clean["Region"].astype("category")
```

```
print("\nUpdated data types:")  
print(df_clean.dtypes)
```

```
print("\nFinal cleaned shape:")  
print(df_clean.shape)
```

Shape: (440, 8)

Missing values:

Channel	0
Region	0
Fresh	0
Milk	0
Grocery	0
Frozen	0
Detergents_Paper	0
Delicassen	0

dtype: int64

Duplicate rows:

0

Negative values:

Fresh	0
Milk	0
Grocery	0
Frozen	0
Detergents_Paper	0
Delicassen	0

dtype: int64

Zero values:

Fresh	0
Milk	0
Grocery	0
Frozen	0
Detergents_Paper	0


```
Delicassen      0
```

```
dtype: int64
```

Channel codes:

```
[np.int64(1), np.int64(2)]
```

Region codes:

```
[np.int64(1), np.int64(2), np.int64(3)]
```

Updated data types:

```
Channel      category
```

```
Region      category
```

```
Fresh      int64
```

```
Milk      int64
```

```
Grocery      int64
```

```
Frozen      int64
```

```
Detergents_Paper  int64
```

```
Delicassen      int64
```

```
dtype: object
```

Final cleaned shape:

```
(440, 8)
```

Exploratory Data Analysis

Exploratory data analysis was conducted to examine the distributions, skewness, potential outliers and relationships between the six spending variables. Spending patterns were also compared across Channel and Region. The findings will be written after reviewing the actual Colab outputs.

```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
spending_columns = [
    "Fresh",
    "Milk",
    "Grocery",
    "Frozen",
    "Detergents_Paper",
    "Delicassen"
]
```

```
# 1. Descriptive statistics
```

```
summary = df_clean[spending_columns].describe().T
summary["Median"] = df_clean[spending_columns].median()
summary["Skewness"] = df_clean[spending_columns].skew()
```

```
print("Descriptive statistics:")
display(summary.round(2))
```

```
# 2. Histograms
```

```
for column in spending_columns:
    plt.figure(figsize=(7, 4))
    plt.hist(df_clean[column], bins=30, edgecolor="black")
    plt.title(f"Distribution of {column} Spending")
    plt.xlabel("Annual Spending")
    plt.ylabel("Number of Customers")
```

```
plt.tight_layout()
plt.show()
```

3. Boxplots

```
for column in spending_columns:
```

```
    plt.figure(figsize=(7, 3))
    plt.boxplot(df_clean[column], vert=False)
    plt.title(f"Boxplot of {column} Spending")
    plt.xlabel("Annual Spending")
    plt.tight_layout()
    plt.show()
```

4. Potential outliers using the IQR method

```
outlier_counts = {}
```

```
for column in spending_columns:
```

```
    q1 = df_clean[column].quantile(0.25)
    q3 = df_clean[column].quantile(0.75)
    iqr = q3 - q1
```

```
    lower_limit = q1 - 1.5 * iqr
```

```
    upper_limit = q3 + 1.5 * iqr
```

```
    outlier_counts[column] = (
        (df_clean[column] < lower_limit) |
        (df_clean[column] > upper_limit)
```

```
).sum()
```

```
outlier_df = pd.DataFrame.from_dict(  
    outlier_counts,  
    orient="index",  
    columns=["Potential Outliers"]  
)
```

```
print("Potential outliers:")  
display(outlier_df)
```

5. Correlation matrix

```
correlation_matrix = df_clean[spending_columns].corr()
```

```
print("Correlation matrix:")  
display(correlation_matrix.round(2))
```

```
plt.figure(figsize=(8, 6))  
plt.imshow(correlation_matrix, aspect="auto")  
plt.colorbar(label="Correlation")
```

```
plt.xticks(  
    range(len(spending_columns)),  
    spending_columns,  
    rotation=45,  
    ha="right"  
)
```

```

plt.yticks(
    range(len(spending_columns)),
    spending_columns
)

for i in range(len(spending_columns)):
    for j in range(len(spending_columns)):
        plt.text(
            j,
            i,
            f"{correlation_matrix.iloc[i, j]:.2f}",
            ha="center",
            va="center"
        )

plt.title("Correlation Between Spending Categories")
plt.tight_layout()
plt.show()

```

6. Median spending by Channel

```

channel_medians = (
    df_clean.groupby("Channel", observed=True)[spending_columns]
    .median()
)

print("Median spending by Channel:")

```

```
display(channel_medians.round(2))
```

```
# 7. Median spending by Region
```

```
region_medians = (  
    df_clean.groupby("Region", observed=True)[spending_columns]  
    .median()  
)
```

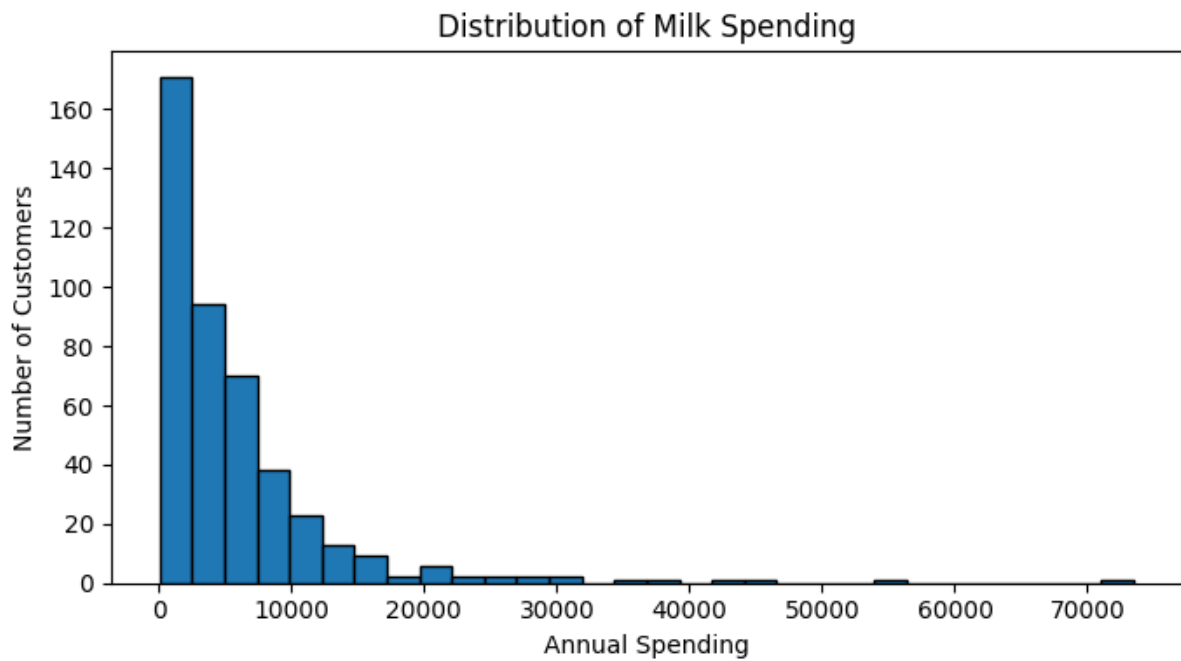
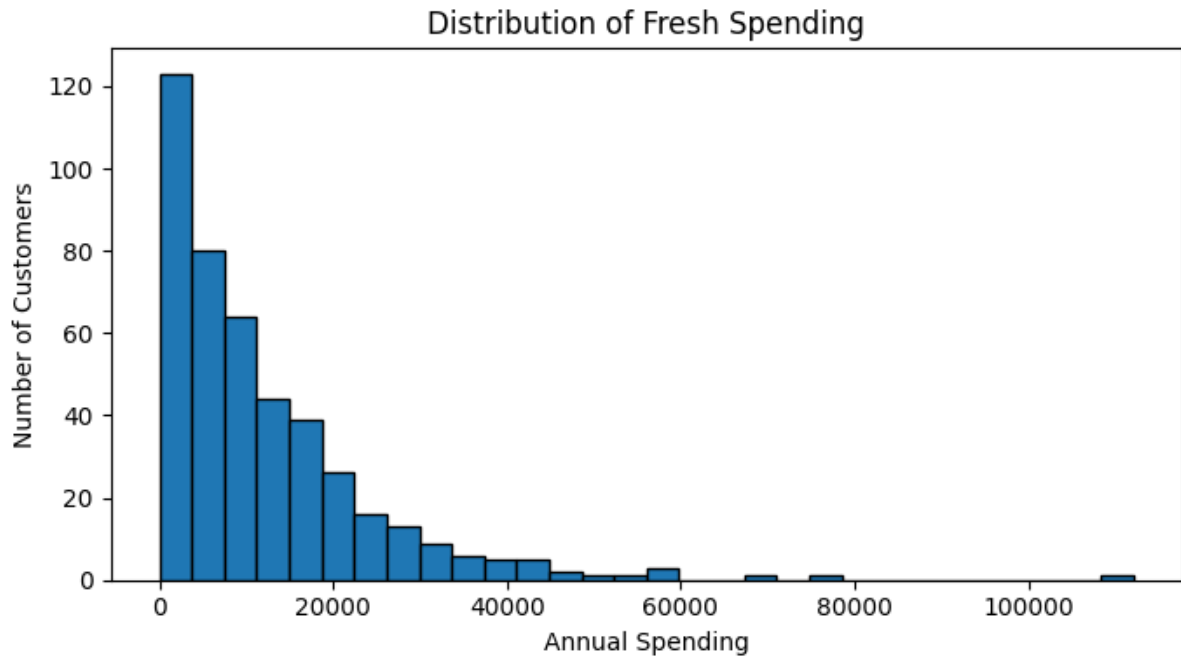
```
print("Median spending by Region:")
```

```
display(region_medians.round(2))
```

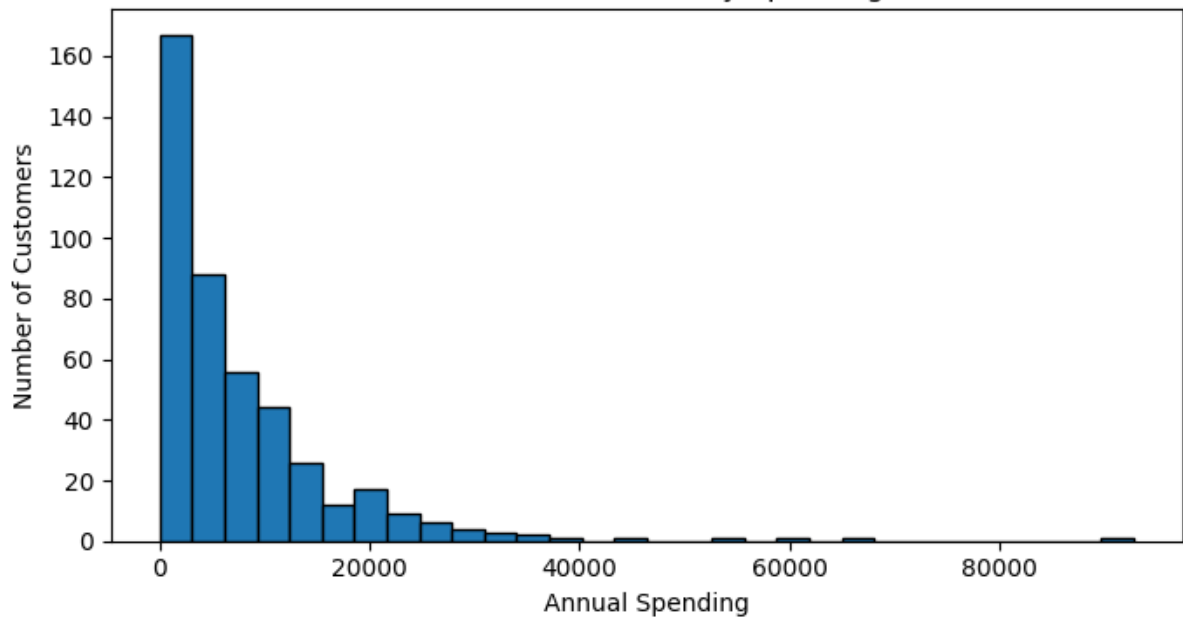
Descriptive statistics:

	count	mean	std	min	25%	50%	75%	max	Median	Skewness
Fresh	440.0	1200.030	1264.733	3.0	3127.75	850.40	1693.75	1121.510	850.40	2.56
Milk	440.0	5796.27	7380.38	55.0	1533.00	362.70	7190.25	7349.80	362.70	4.05
Grocery	440.0	7951.28	9503.16	3.0	2153.00	475.55	1065.75	9278.00	475.55	3.59
Frozen	440.0	3071.93	4854.67	25.0	742.25	152.60	3554.25	6086.90	152.60	5.91
Detergents_Paper	440.0	2881.49	4767.85	3.0	256.75	816.5	3922.00	4082.70	816.5	3.63

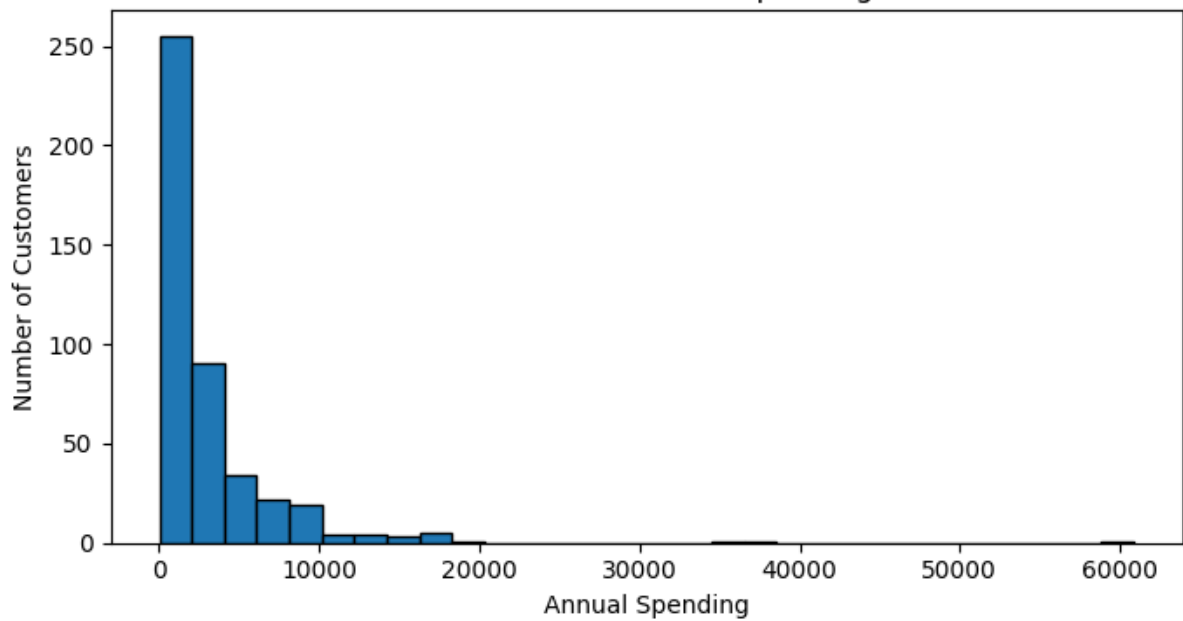
	count	mean	std	min	25%	50%	75%	max	Median	Skewness
Delicasse	440	1524.	2820.	3.	408.	965	1820.	4794	965.	11.15
n	.0	87	11	0	25	.5	25	3.0	5	



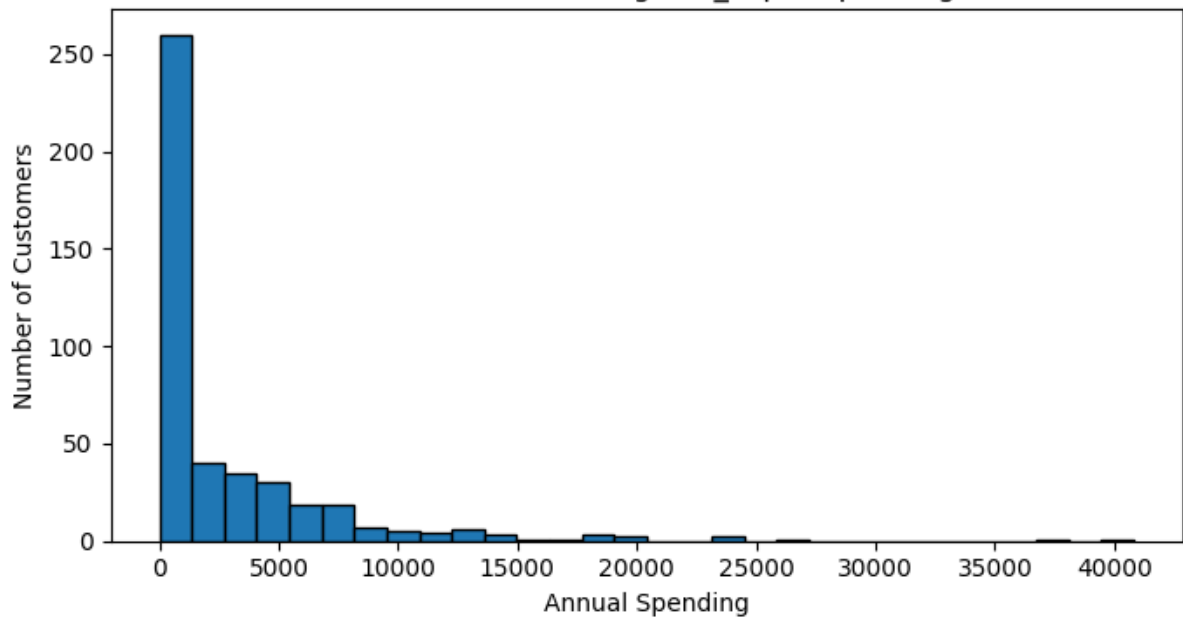
Distribution of Grocery Spending



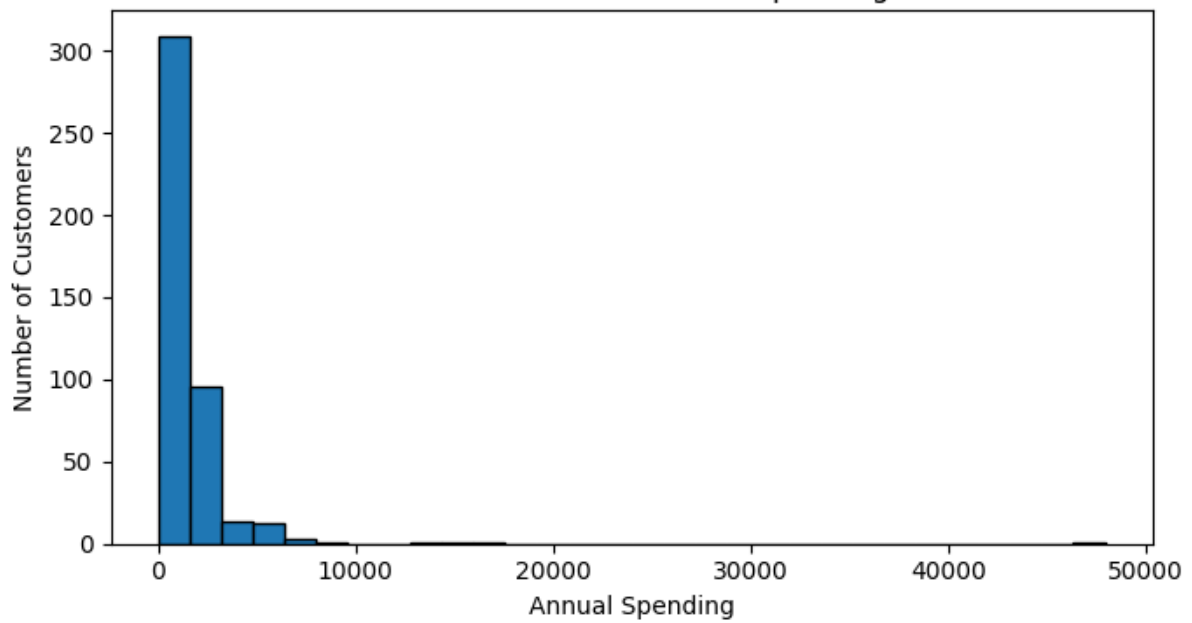
Distribution of Frozen Spending



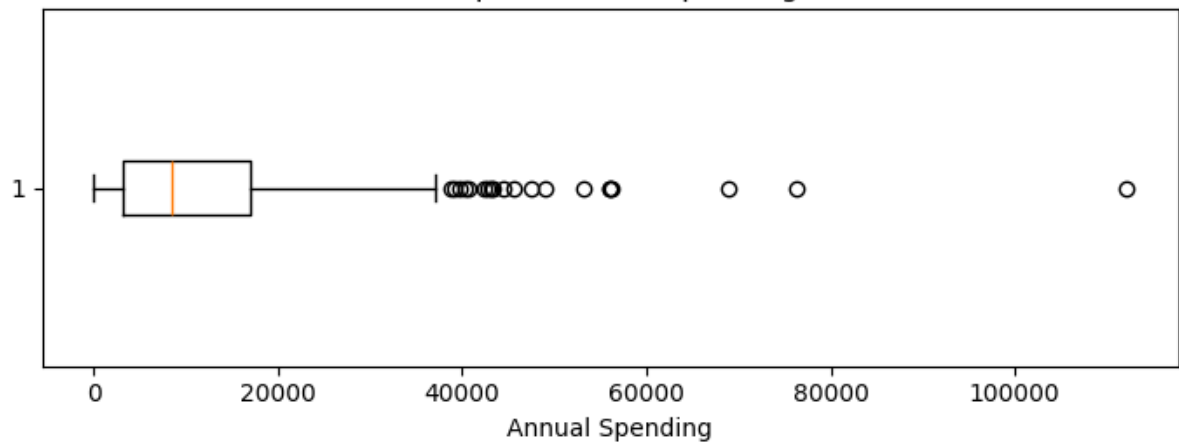
Distribution of Detergents_Paper Spending



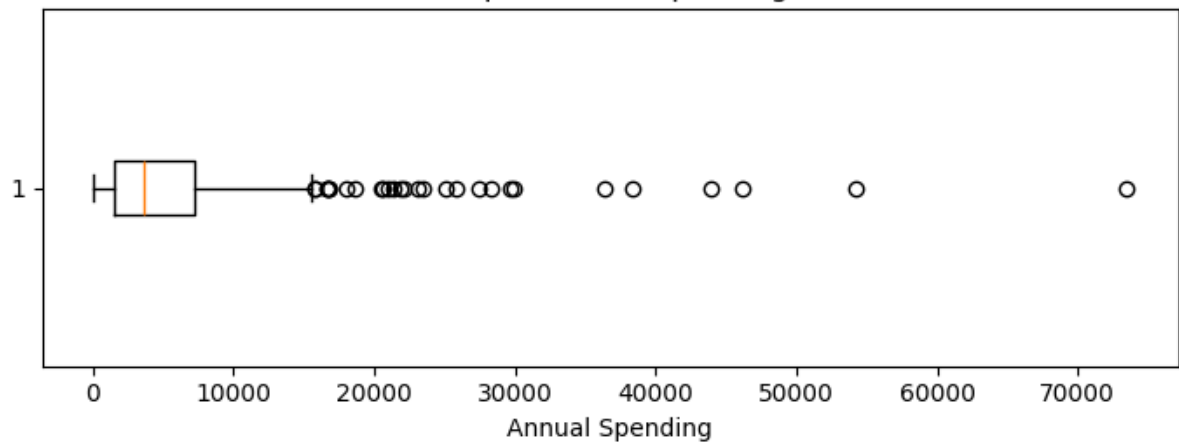
Distribution of Delicassen Spending



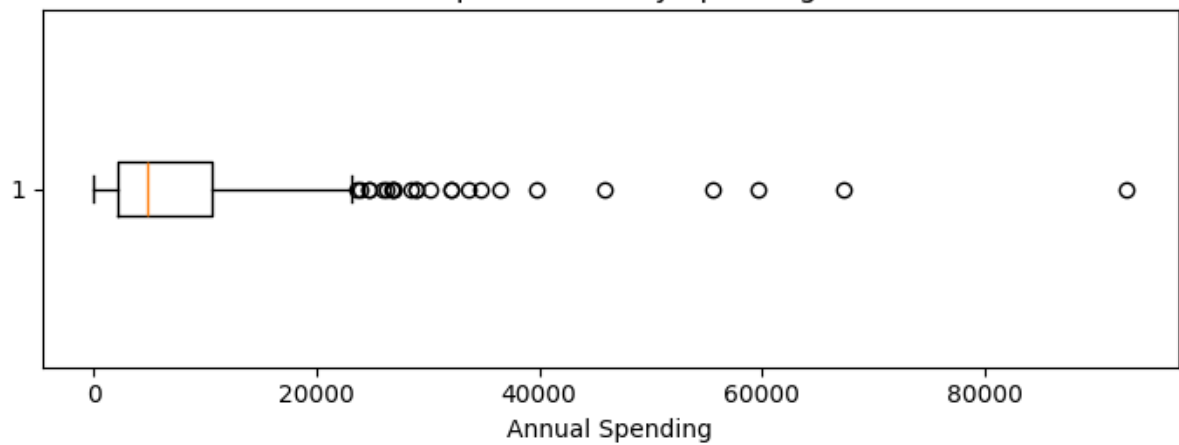
Boxplot of Fresh Spending

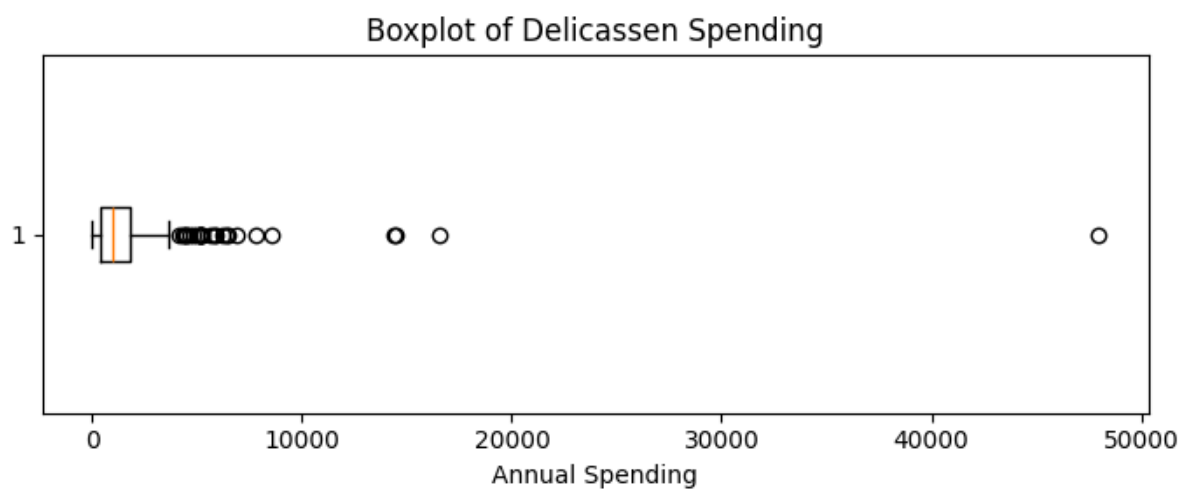
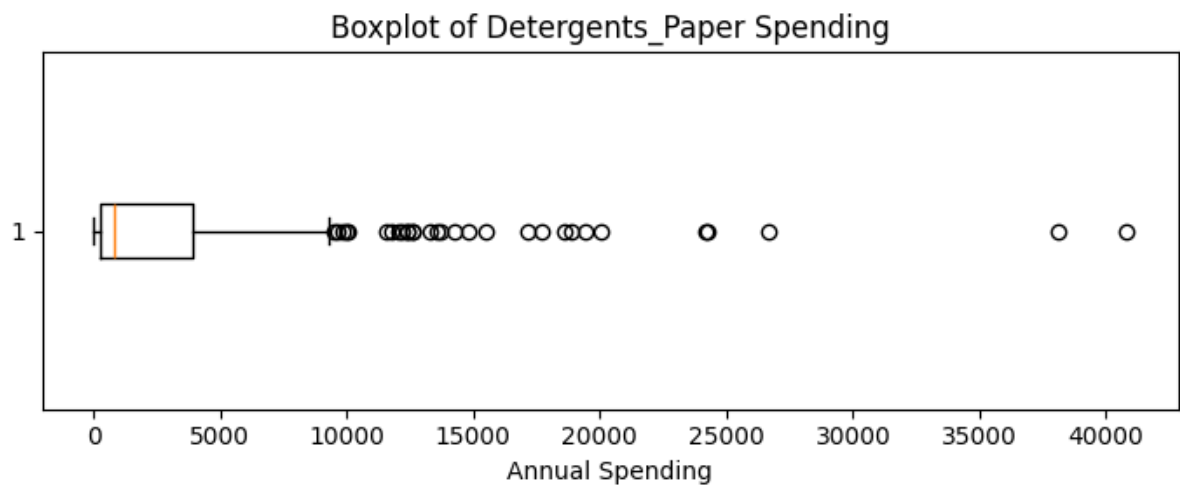
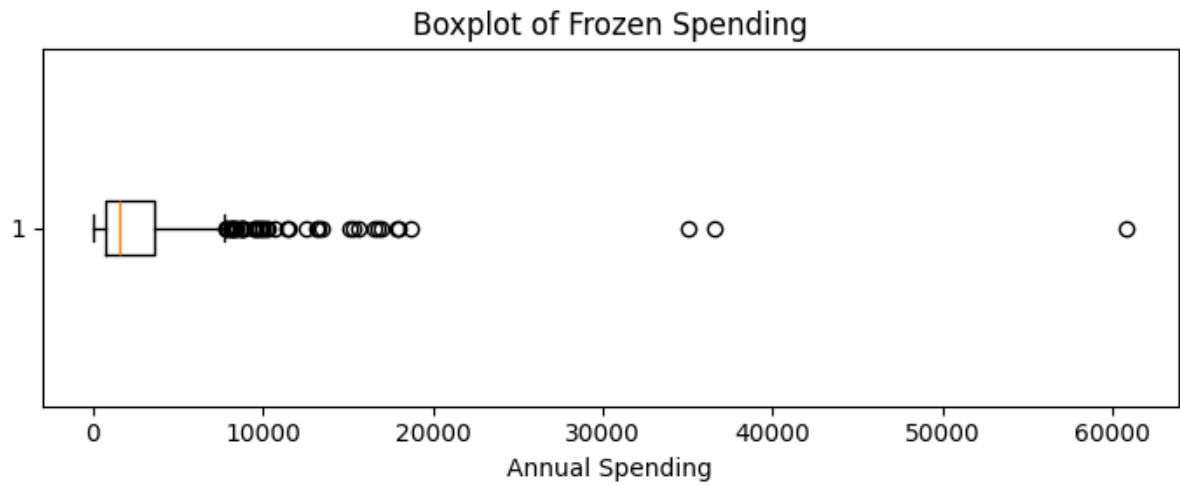


Boxplot of Milk Spending



Boxplot of Grocery Spending



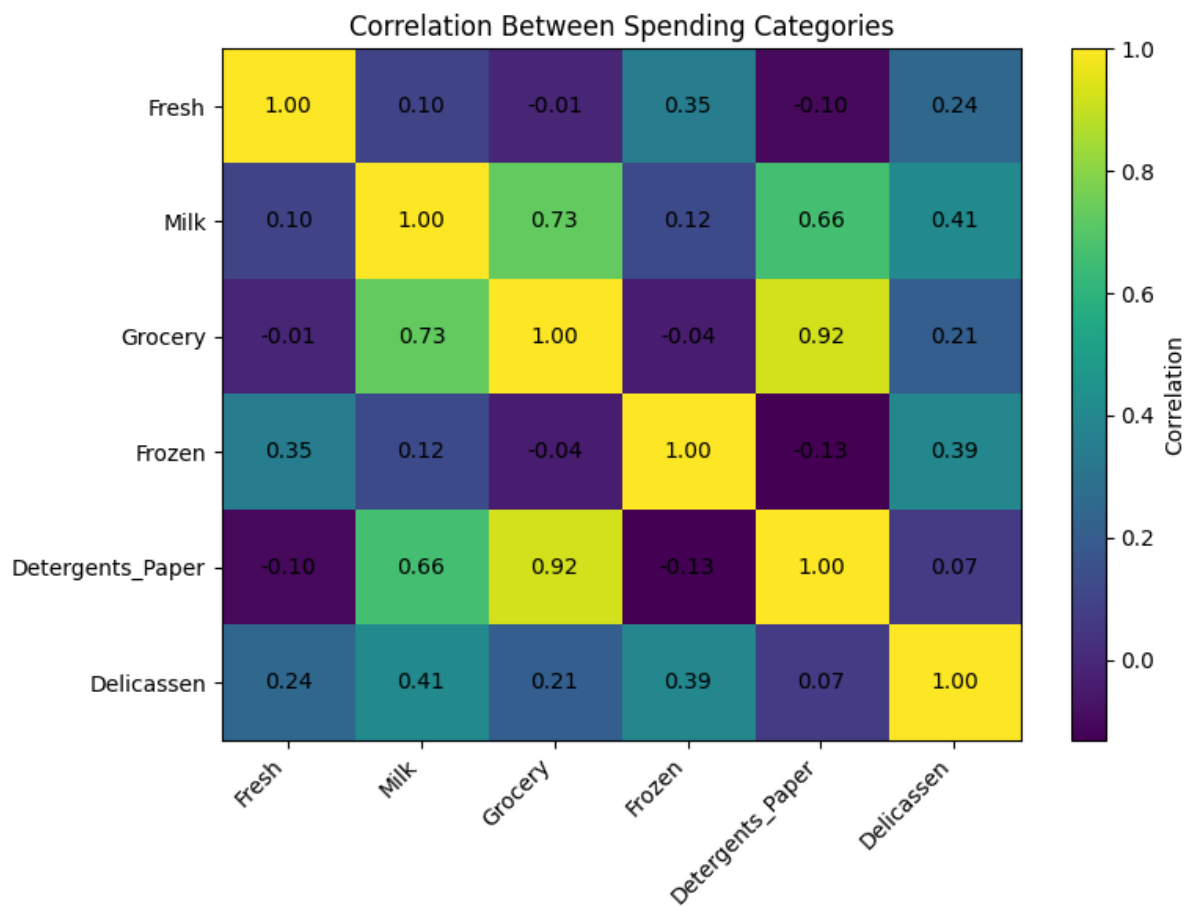


Potential outliers:

Potential Outliers	
Fresh	20
Milk	28
Grocery	24
Frozen	43
Detergents_Paper	30
Delicassen	27

Correlation matrix:

	Fres h	Mil k	Grocer y	Froze n	Detergents_Pap er	Delicasse n
Fresh	1.00	0.10	-0.01	0.35	-0.10	0.24
Milk	0.10	1.00	0.73	0.12	0.66	0.41
Grocery	-0.01	0.73	1.00	-0.04	0.92	0.21
Frozen	0.35	0.12	-0.04	1.00	-0.13	0.39
Detergents_Paper	-0.10	0.66	0.92	-0.13	1.00	0.07
Delicassen	0.24	0.41	0.21	0.39	0.07	1.00



Median spending by Channel:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
Channel						
1	9581.5	2157.0	2684.0	2057.5	385.5	821.0
2	5993.5	7812.0	12390.0	1081.0	5614.5	1350.0

Median spending by Region:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
Region						
1	7363.0	3748.0	3838.0	1801.0	737.0	806.0

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
Region						
2	8090.0	2374.0	6114.0	1455.0	811.0	898.0
3	8752.5	3684.5	4732.0	1498.0	856.0	994.0

All six spending variables were positively skewed, with mean spending higher than median spending. Fresh had the highest average annual spending at approximately **12,000**, followed by Grocery and Milk. Delicassen and Frozen showed the strongest skewness, indicating that a small number of customers had extremely high spending.

The IQR method identified potential outliers in every category, particularly Frozen with **43 potential outliers**. These values were retained because they may represent genuine high-value customers.

The strongest correlation was between **Grocery and Detergents_Paper (0.92)**, followed by Milk and Grocery (0.73). Channel showed clearer spending differences than Region. Channel 1 customers generally spent more on Fresh and Frozen products, while Channel 2 customers spent considerably more on Milk, Grocery and Detergents_Paper. Regional differences were comparatively smaller.

3.3 Feature Engineering and Feature Selection

Additional features were created to better describe customer purchasing behaviour. These included total spending, average spending, spending proportions, the dominant product category and product diversity. These features will mainly help interpret the final clusters.

For clustering, the six original spending variables were selected. Channel and Region were excluded because they are categorical and will instead be used to compare the completed clusters. Total Spending was not added to the clustering inputs because it is calculated from the six spending variables and would create unnecessary duplication.

```
# Create a feature-engineered copy
```

```
df_features = df_clean.copy()
```

```
# Total and average annual spending
```

```
df_features["Total_Spending"] = df_features[spending_columns].sum(axis=1)
df_features["Average_Spending"] = df_features[spending_columns].mean(axis=1)
```

```
# Spending proportion for each category
```

```
share_columns = []
```

```
for column in spending_columns:
```

```
    share_name = f"{column}_Share"
```

```
    df_features[share_name] = (
```

```
        df_features[column] / df_features["Total_Spending"]
```

```
    )
```

```
    share_columns.append(share_name)
```

```
# Customer's highest-spending category
```

```
df_features["Dominant_Category"] = (
```

```
    df_features[spending_columns].idxmax(axis=1)
```

```
)
```

```
# Number of categories representing at least 10% of total spending
```

```
df_features["Product_Diversity"] = (
```

```
    df_features[share_columns] >= 0.10
```

```
).sum(axis=1)
```

```
print("Engineered features:")
```

```
display(
```

```
    df_features[
```

```
        [
```

```
            "Total_Spending",
```

```

        "Average_Spending",
        "Dominant_Category",
        "Product_Diversity"
    ] + share_columns
].head()
)

# Check for zero-variance features
feature_variance = df_features[spending_columns].var()

print("\nFeature variance:")
display(feature_variance.to_frame(name="Variance"))

# Identify highly correlated feature pairs
correlation_matrix = df_features[spending_columns].corr()
high_correlation_pairs = []

for i in range(len(spending_columns)):
    for j in range(i + 1, len(spending_columns)):
        correlation = correlation_matrix.iloc[i, j]

        if abs(correlation) >= 0.90:
            high_correlation_pairs.append({
                "Feature 1": spending_columns[i],
                "Feature 2": spending_columns[j],
                "Correlation": round(correlation, 3)
            })

```



```
print("\nHighly correlated feature pairs:")
display(pd.DataFrame(high_correlation_pairs))
```

```
# Final variables selected for clustering
selected_features = spending_columns.copy()
```

```
print("\nSelected clustering features:")
print(selected_features)
```

Engineered features:

	Total _Spendin g	Avera ge_Sp endin g	Domin ant_C ategor y	Produ ct_Di versit y	Fres h_S har e	Mil k_S har e	Groc ery_ Shar e	Froz en_ Shar e	Deterge nts_Pap er_Shar e	Delic assen _Shar e
0	34112	5685.333333	Fresh	3	0.371394	0.283068	0.221652	0.006273	0.078389	0.039224
1	33266	5544.333333	Milk	3	0.212139	0.294896	0.287621	0.052967	0.098990	0.053388
2	36610	6101.666667	Milk	4	0.173532	0.240590	0.209888	0.065692	0.096039	0.214258
3	27381	4563.500000	Fresh	3	0.484460	0.043680	0.154158	0.233885	0.018516	0.065301
4	46100	7683.333333	Fresh	4	0.490564	0.117354	0.156139	0.084924	0.038547	0.112473

Feature variance:

	Variance
Fresh	1.599549e+08
Milk	5.446997e+07
Grocery	9.031010e+07
Frozen	2.356785e+07
Detergents_Paper	2.273244e+07
Delicassen	7.952997e+06

Highly correlated feature pairs:

	Feature 1	Feature 2	Correlation
0	Grocery	Detergents_Paper	0.925

Selected clustering features:

['Fresh', 'Milk', 'Grocery', 'Frozen', 'Detergents_Paper', 'Delicassen']

Total spending, average spending, category spending shares, dominant product category and product diversity were created to help describe and interpret customer purchasing behaviour.

Fresh was the dominant spending category for **246 customers**, while Grocery was dominant for **126 customers**. Most customers spent at least 10% of their total spending across three or four product categories, suggesting that many customers purchased a reasonably diverse product range.

The six original spending variables were retained for clustering because they directly represent customer purchasing behaviour. Channel and Region were excluded from the clustering inputs and will instead be used to interpret the resulting customer segments. Total Spending was also excluded because it is calculated from the original variables and would duplicate information.

3.4 Data Transformation and Dimensionality Reduction

The six spending variables were transformed using log1p to reduce skewness and the influence of extreme values. Standardisation was then applied so that each feature contributed equally to clustering. PCA was used to reduce the transformed data to two components for visualisation.

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

# Select the six spending features

X = df_features[selected_features].copy()

# Log transformation to reduce skewness

X_log = np.log1p(X)

# Standardise the transformed features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_log)
```

```

# Store scaled data as a DataFrame
X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=selected_features,
    index=df_features.index
)

print("Scaled feature preview:")
display(X_scaled_df.head())

# Apply PCA using all components
pca_full = PCA()
pca_full.fit(X_scaled)

explained_variance = pd.DataFrame({
    "Principal Component": [
        f"PC{i + 1}" for i in range(len(selected_features))
    ],
    "Explained Variance Ratio": pca_full.explained_variance_ratio_,
    "Cumulative Variance": np.cumsum(
        pca_full.explained_variance_ratio_
    )
})

print("\nPCA explained variance:")
display(explained_variance.round(4))

# Plot cumulative explained variance

```

```
plt.figure(figsize=(7, 4))

plt.plot(
    range(1, len(selected_features) + 1),
    explained_variance["Cumulative Variance"],
    marker="o"
)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("PCA Cumulative Explained Variance")
plt.xticks(range(1, len(selected_features) + 1))
plt.grid()
plt.tight_layout()
plt.show()
```

```
# Reduce the data to two principal components
```

```
pca_2d = PCA(n_components=2)
```

```
X_pca = pca_2d.fit_transform(X_scaled)
```

```
pca_df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"],
    index=df_features.index
)
```

```
# Retain Channel and Region for later interpretation
```

```
pca_df["Channel"] = df_features["Channel"].values
```

```
pca_df["Region"] = df_features["Region"].values
```

```
print("\nTwo-dimensional PCA data:")

display(pca_df.head())


print(
    "\nVariance explained by the first two components:",
    round(pca_2d.explained_variance_ratio_.sum(), 4)
)


# PCA loadings
pca_loadings = pd.DataFrame(
    pca_2d.components_.T,
    index=selected_features,
    columns=["PC1", "PC2"]
)


print("\nPCA feature loadings:")
display(pca_loadings.round(3))

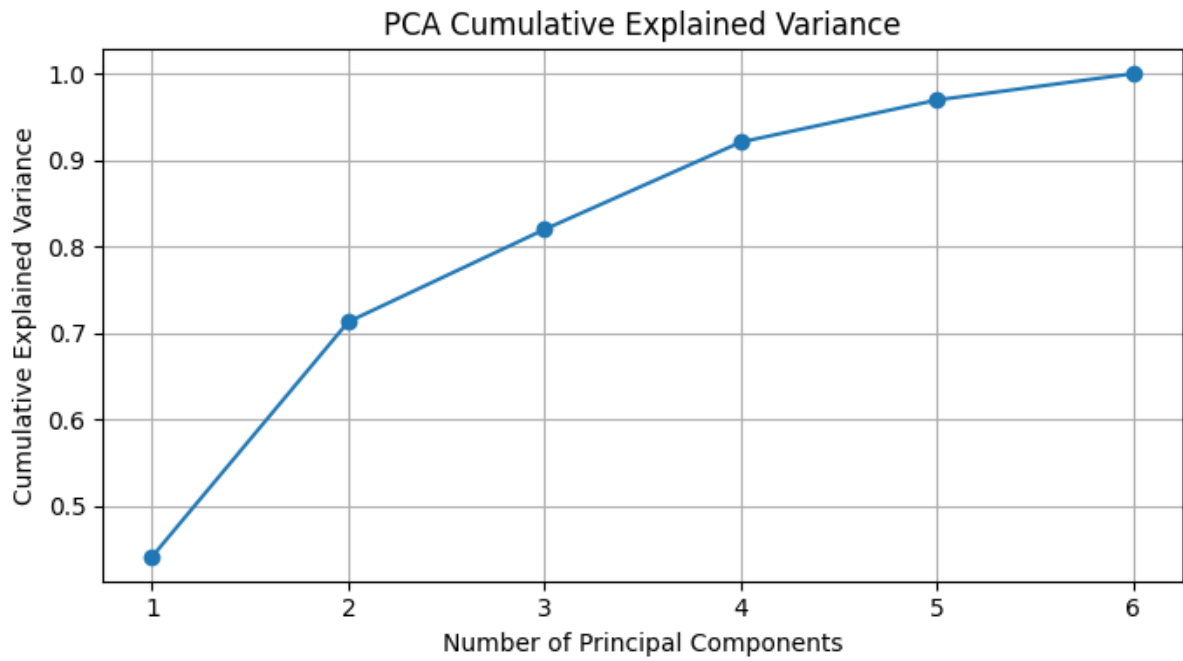

# PCA visualisation
plt.figure(figsize=(7, 5))
plt.scatter(pca_df["PC1"], pca_df["PC2"], alpha=0.7)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualisation of Wholesale Customers")
plt.tight_layout()
plt.show()
```

Scaled feature preview:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	0.486184	0.976299	0.440155	- 1.509250	0.644143	0.408966
1	0.087889	0.990956	0.652171	0.134052	0.766043	0.627926
2	0.016356	0.891151	0.454687	0.376899	0.804405	1.776833
3	0.517477	- 0.957973	- 0.084792	1.141574	-0.328712	0.633133
4	0.880631	0.439662	0.395847	0.757322	0.404939	1.456588

PCA explained variance:

	Principal Component	Explained Variance Ratio	Cumulative Variance
0	PC1	0.4408	0.4408
1	PC2	0.2719	0.7127
2	PC3	0.1070	0.8197
3	PC4	0.1010	0.9207
4	PC5	0.0488	0.9695
5	PC6	0.0305	1.0000



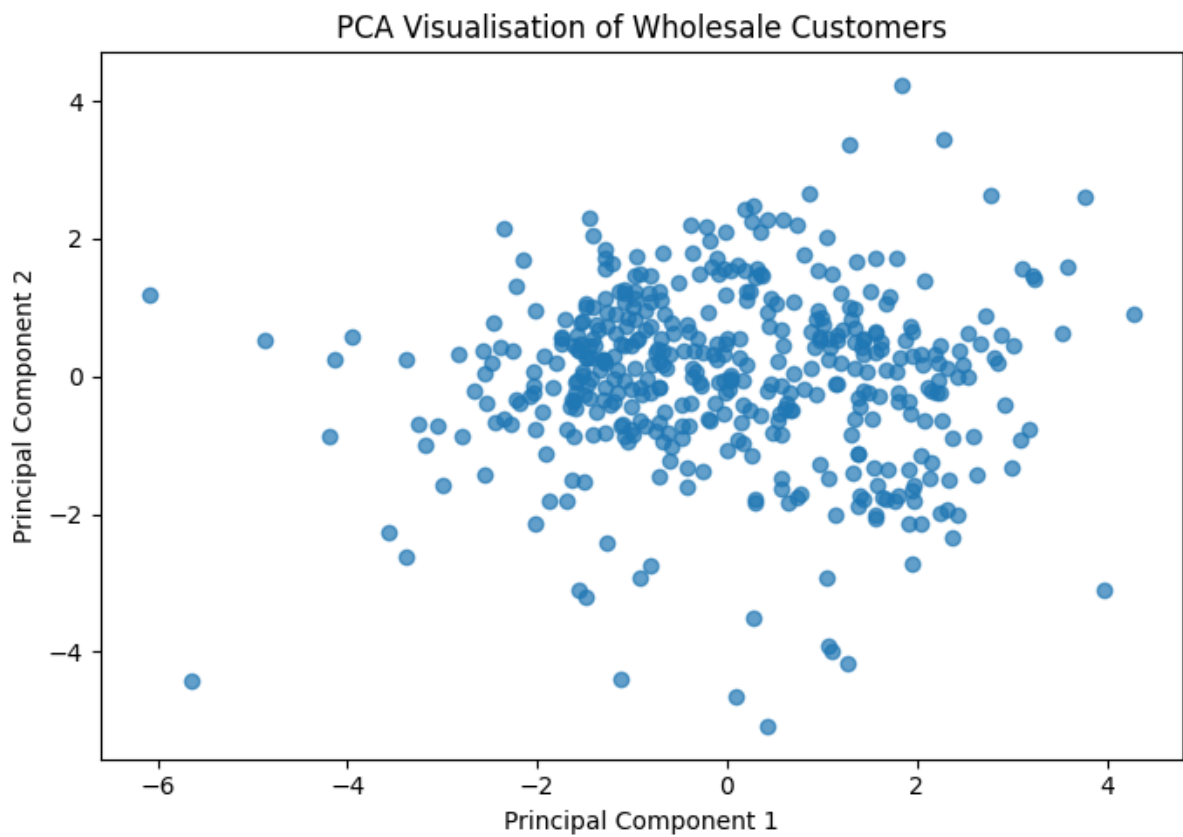
Two-dimensional PCA data:

	PC1	PC2	Channel	Region
0	1.380383	-0.304727	2	3
1	1.438870	0.537468	2	3
2	1.513453	1.232565	2	3
3	-0.825105	1.210790	1	3
4	0.802897	1.766046	2	3

Variance explained by the first two components: 0.7127

PCA feature loadings:

	PC1	PC2
Fresh	-0.105	0.590
Milk	0.542	0.133
Grocery	0.571	-0.006
Frozen	-0.138	0.590
Detergents_Paper	0.551	-0.071
Delicassen	0.214	0.530



A log1p transformation was applied to reduce the influence of highly skewed spending values and extreme customers. StandardScaler was then used to place all six spending variables on a comparable scale.

The first principal component explained **44.08%** of the total variance, while the second explained **27.19%**. Together, the first two components explained approximately **71.27%** of the dataset's variation.

PC1 was mainly influenced by Milk, Grocery and Detergents_Paper spending, while PC2 was mainly influenced by Fresh, Frozen and Delicassen spending. The PCA results indicate meaningful structure within the data, although clustering models are still required to formally identify customer segments.

Step 4: Model Implementation and Comparison

Three clustering models will be tested:

1. K-Means
2. Hierarchical Clustering
3. DBSCAN

The models will then be compared using the Silhouette Score, Davies–Bouldin Index and Calinski–Harabasz Score.

K-Means

K-Means is an unsupervised learning method that divides customers into a chosen number of clusters based on similarity. Each customer is assigned to the nearest cluster centre, and the centres are repeatedly updated until the groups stabilise.

K-means will be tested with different numbers of clusters. The elbow method and evaluation metrics will help select the most suitable value of **K**.

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.metrics import (
```

```
    silhouette_score,
```

```
    davies_bouldin_score,
```

```
calinski_harabasz_score
)

# Load dataset
df = pd.read_csv("/content/Wholesale customers data.csv")

# Select spending features
selected_features = [
    "Fresh",
    "Milk",
    "Grocery",
    "Frozen",
    "Detergents_Paper",
    "Delicassen"
]

X = df[selected_features].copy()

# Log transformation
X_log = np.log1p(X)

# Standardisation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_log)

X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=selected_features,
```

```

        index=df.index
    )

# Test K-Means from 2 to 10 clusters
kmeans_results = []

for k in range(2, 11):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X_scaled_df)

    kmeans_results.append({
        "Clusters": k,
        "Inertia": model.inertia_,
        "Silhouette Score": silhouette_score(X_scaled_df, labels),
        "Davies-Bouldin Index": davies_bouldin_score(
            X_scaled_df, labels
        ),
        "Calinski-Harabasz Score": calinski_harabasz_score(
            X_scaled_df, labels
        )
    })

kmeans_results_df = pd.DataFrame(kmeans_results)

```

```
display(kmeans_results_df.round(3))
```

	Clusters	Inertia	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
0	2	1844.064	0.290	1.352	189.050
1	3	1553.421	0.259	1.341	152.835
2	4	1386.863	0.188	1.541	131.320
3	5	1266.149	0.192	1.484	118.000
4	6	1174.759	0.201	1.432	108.263
5	7	1086.359	0.196	1.392	103.208
6	8	1028.486	0.185	1.441	96.699
7	9	973.141	0.197	1.383	92.281
8	10	931.998	0.191	1.344	87.559

The K-Means model was tested using **2 to 10 clusters**. The highest Silhouette Score was achieved with **2 clusters at 0.290**, indicating that the two-cluster solution produced the clearest separation between customers.

The two-cluster model also achieved the highest Calinski–Harabasz Score of **189.050**, supporting stronger cluster separation and compactness compared with the other values of K.

Although the three-cluster model produced the lowest Davies–Bouldin Index of **1.341**, this was only slightly better than the two-cluster result of **1.352**. Its Silhouette and Calinski–Harabasz scores were also lower.

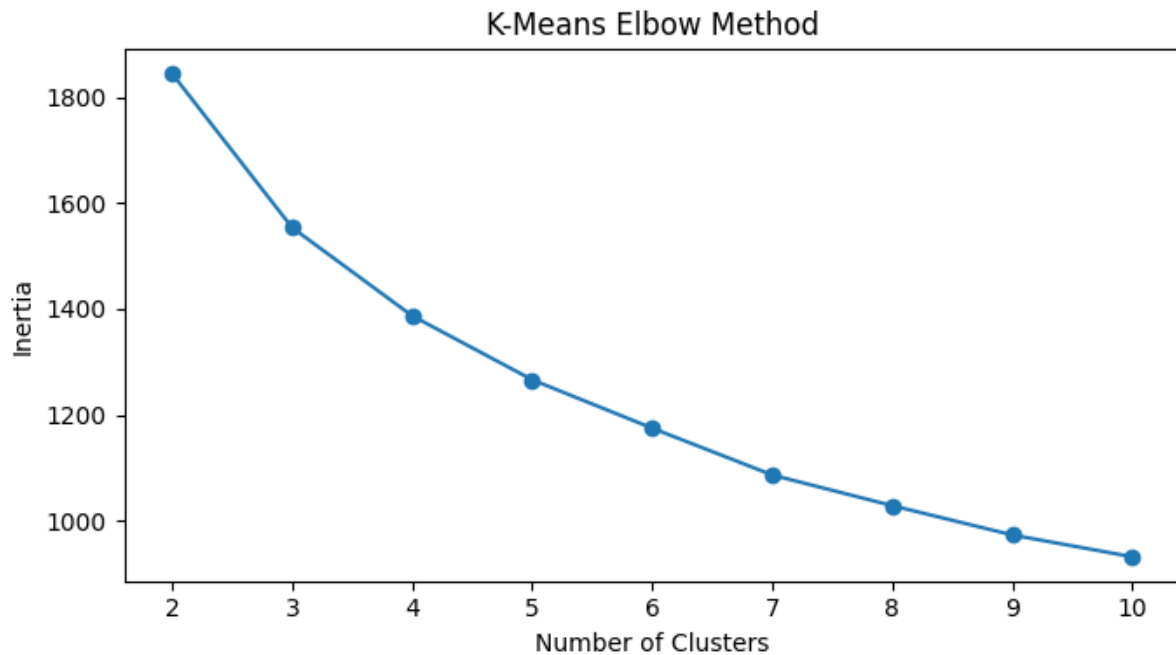
Therefore, based on the combined evaluation metrics, **K = 2 is currently the strongest K-Means solution**. However, the relatively low Silhouette Score suggests that the customer groups overlap and are not perfectly separated.

Elbow plot

```
plt.figure(figsize=(7, 4))

plt.plot(
    kmeans_results_df["Clusters"],
    kmeans_results_df["Inertia"],
    marker="o"
)

plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("K-Means Elbow Method")
plt.xticks(range(2, 11))
plt.tight_layout()
plt.show()
```



The inertia decreases as the number of clusters increases, which is expected. The largest reduction occurs between **2 and 3 clusters**, after which the decline becomes more gradual.

This suggests that the elbow is around **3 clusters**, although it is not extremely clear. Therefore, the elbow method slightly favours **K = 3**, while the Silhouette and Calinski–Harabasz scores favour **K = 2**.

Both two- and three-cluster solutions should therefore be considered before selecting the final K-Means model.

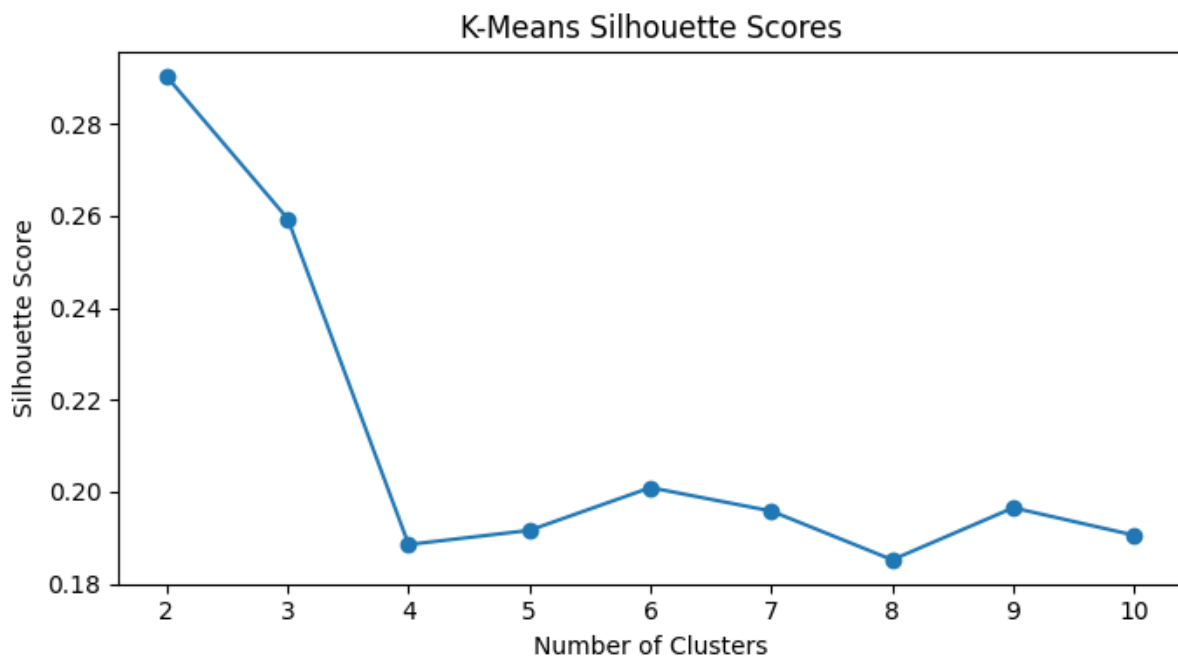
Silhouette Plot

```
plt.figure(figsize=(7, 4))
```

```
plt.plot(  
    kmeans_results_df["Clusters"],  
    kmeans_results_df["Silhouette Score"],  
    marker="o"  
)
```

```
plt.xlabel("Number of Clusters")
```

```
plt.ylabel("Silhouette Score")
plt.title("K-Means Silhouette Scores")
plt.xticks(range(2, 11))
plt.tight_layout()
plt.show()
```



The highest Silhouette Score was achieved at **K = 2**, with a score of approximately **0.290**. This indicates that the two-cluster solution provides the best separation and internal consistency among the K-Means models tested.

The score for **K = 3** was lower at approximately **0.259**, while the scores dropped noticeably from K = 4 onwards.

Therefore, the Silhouette analysis supports selecting **two clusters**. However, the overall score is still relatively low, suggesting that the customer groups overlap and are not strongly separated.

The best-performing K-means model

Select K with the highest Silhouette Score

```
best_k = int(
    kmeans_results_df.loc[
```



```

        kmeans_results_df["Silhouette Score"].idxmax(),
        "Clusters"
    ]
)

print("Best K based on Silhouette Score:", best_k)

final_kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

kmeans_labels = final_kmeans.fit_predict(X_scaled_df)

# Using 'df' as 'df_features' was not defined in the current session
df["KMeans_Cluster"] = kmeans_labels

print("\nCluster sizes:")

display(
    df["KMeans_Cluster"]
    .value_counts()
    .sort_index()
    .to_frame("Customers")
)

```

Best K based on Silhouette Score: 2

Cluster sizes:

	Customers
KMeans_Cluster	
0	252
1	188

Based on the highest Silhouette Score, the best-performing K-Means model used **two clusters**. Cluster 0 contained **252 customers**, while Cluster 1 contained **188 customers**. This suggests that the model produced two reasonably balanced customer groups, although the cluster labels 0 and 1 are arbitrary and do not yet describe the characteristics of each group.

Visualise the clusters using PCA

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
from sklearn.decomposition import PCA
```

```
# Fix: Check if pca_df exists, otherwise recreate it from X_scaled
```

```
try:
```

```
    _ = pca_df
```

```
except NameError:
```

```
    pca_2d = PCA(n_components=2)
```

```
    X_pca = pca_2d.fit_transform(X_scaled)
```

```
    pca_df = pd.DataFrame(
```

```
        X_pca,
```

```
        columns=["PC1", "PC2"],
```

```
        index=range(len(X_pca))
```

```
)
```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(  
    pca_df["PC1"],  
    pca_df["PC2"],  
    c=kmeans_labels,  
    cmap='viridis',  
    alpha=0.7  
)
```

```
plt.xlabel("Principal Component 1")
```

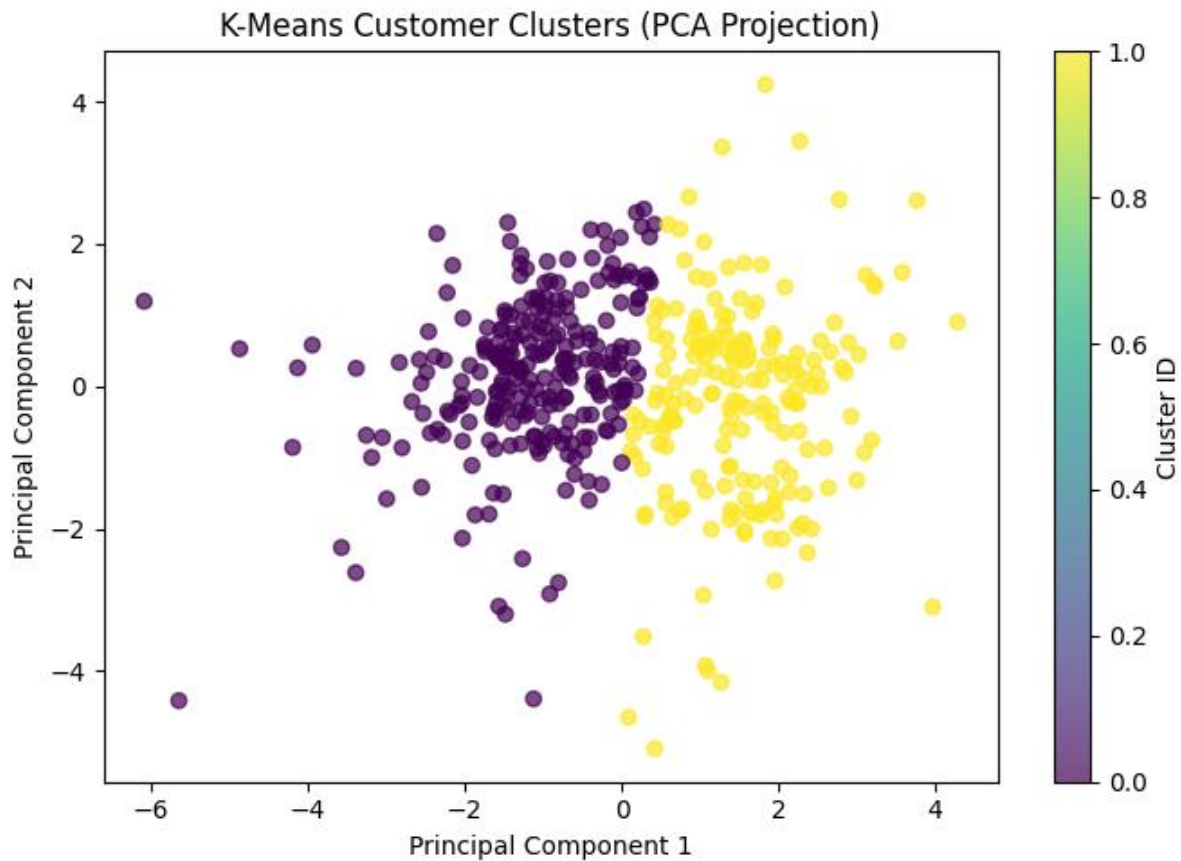
```
plt.ylabel("Principal Component 2")
```

```
plt.title("K-Means Customer Clusters (PCA Projection)")
```

```
plt.colorbar(label='Cluster ID')
```

```
plt.tight_layout()
```

```
plt.show()
```



The PCA plot shows that the two K-Means clusters are mainly separated along **Principal Component 1**. Cluster 0 is concentrated on the left side of the plot, while Cluster 1 is mostly located on the right.

There is some overlap near the centre, which supports the relatively low Silhouette Score of **0.290**. This means the two customer groups are distinguishable, but the separation is not perfect.

Overall, the plot supports the use of two clusters, while also showing that some customers have similar spending patterns across both groups.

Customer spending profile by cluster

```
kmeans_profiles = (
    df
    .groupby("KMeans_Cluster")[selected_features]
    .median()
)
```

```
print("Median spending profile by cluster:")
display(kmeans_profiles.round(2))
```

Median spending profile by cluster:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
KMeans_Cluster						
0	10347.5	1871.0	2406.0	2225.0	327.5	749.5
1	5406.5	7690.5	11527.0	1061.5	4495.0	1388.5

The median spending profiles show two clearly different customer groups:

- **Cluster 0:** Higher spending on **Fresh** and **Frozen** products, but much lower spending on **Milk**, **Grocery**, **Detergents_Paper** and **Delicassen**. This cluster can be described as **Fresh and Frozen-focused customers**.
- **Cluster 1:** Much higher spending on **Milk**, **Grocery**, **Detergents_Paper** and **Delicassen**, but lower spending on **Fresh** and **Frozen**. This cluster can be described as **Grocery and household-product-focused customers**.

These differences make the two clusters meaningful from a business perspective because each group would likely require different sales, promotion and cross-selling strategies.

Hierarchical Clustering

Hierarchical clustering gradually combines similar customers into larger groups and displays the process through a dendrogram. Unlike K-Means, it does not require the final number of clusters to be selected before building the hierarchy.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage, dendrogram

# Load dataset
df = pd.read_csv("/content/Wholesale customers data.csv")

selected_features = [
    "Fresh",
    "Milk",
    "Grocery",
    "Frozen",
    "Detergents_Paper",
    "Delicassen"
]

# Prepare data
X = df[selected_features].copy()
X_log = np.log1p(X)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_log)

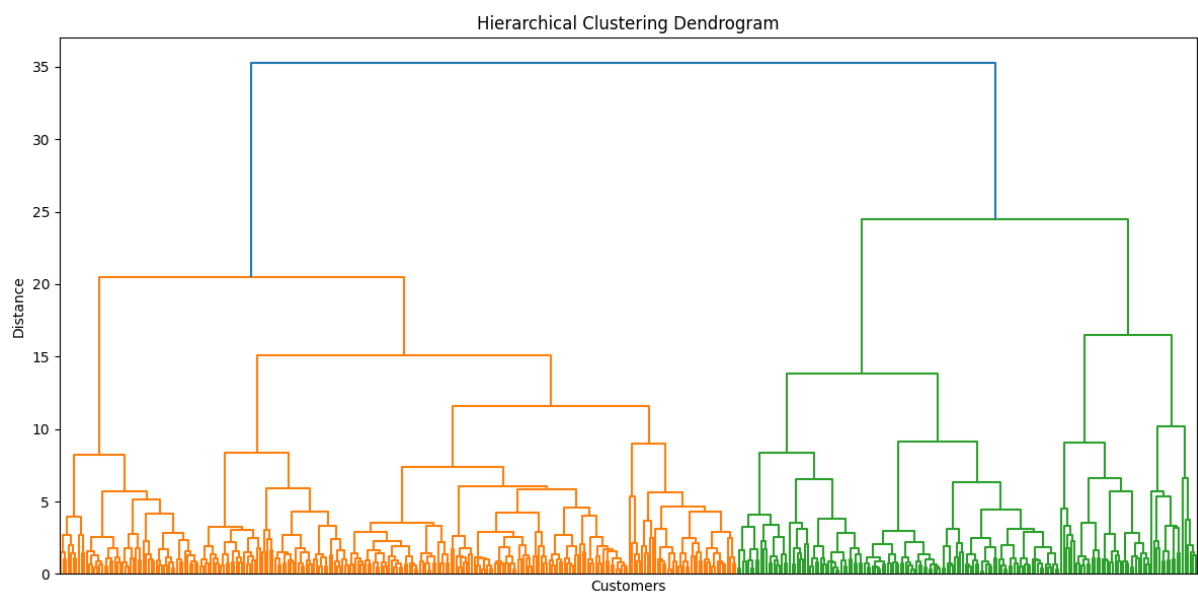
X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=selected_features,
    index=df.index
)
```

```
# Create linkage matrix using Ward linkage  
linkage_matrix = linkage(X_scaled_df, method="ward")
```

```
# Plot dendrogram  
plt.figure(figsize=(12, 6))
```

```
dendrogram(  
    linkage_matrix,  
    no_labels=True  
)
```

```
plt.title("Hierarchical Clustering Dendrogram")  
plt.xlabel("Customers")  
plt.ylabel("Distance")  
plt.tight_layout()  
plt.show()
```



Hierarchical Clustering Evaluation

Now we will test 2 to 10 clusters using Ward Linkage and compare the same three evaluation metrics used for K-Means.

```
from sklearn.cluster import AgglomerativeClustering

from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score
)

hierarchical_results = []

for n_clusters in range(2, 11):

    model = AgglomerativeClustering(
        n_clusters=n_clusters,
        linkage="ward"
    )

    labels = model.fit_predict(X_scaled_df)

    hierarchical_results.append({
        "Clusters": n_clusters,
        "Silhouette Score": silhouette_score(X_scaled_df, labels),
        "Davies-Bouldin Index": davies_bouldin_score(
            X_scaled_df, labels
        ),
```



```

"Calinski-Harabasz Score": calinski_harabasz_score(
    X_scaled_df, labels
)
})

```

```
hierarchical_results_df = pd.DataFrame(hierarchical_results)
```

```
print("Hierarchical clustering evaluation results:")
```

```
display(hierarchical_results_df.round(3))
```

Hierarchical clustering evaluation results:

	Clusters	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
0	2	0.258	1.600	134.624
1	3	0.255	1.539	116.799
2	4	0.202	1.521	108.607
3	5	0.203	1.386	100.015
4	6	0.171	1.490	94.857
5	7	0.163	1.498	91.301
6	8	0.148	1.587	86.612
7	9	0.155	1.549	82.006
8	10	0.122	1.501	77.715

The **two-cluster solution** performed best overall:

- Highest Silhouette Score: **0.258**
- Highest Calinski–Harabasz Score: **134.624**
- Davies–Bouldin Index: **1.600**

Although the five-cluster model had the lowest Davies–Bouldin Index of **1.386**, its Silhouette and Calinski–Harabasz scores were lower.

Therefore, **two clusters** should be selected for the hierarchical model. This is also consistent with the dendrogram, which showed two main customer groups. However, hierarchical clustering performed slightly worse than K-Means, which achieved a Silhouette Score of **0.290**.

Hierarchical Cluster Size

```
from sklearn.cluster import AgglomerativeClustering

import matplotlib.pyplot as plt

# Fit final hierarchical model

final_hierarchical = AgglomerativeClustering(

    n_clusters=2,

    linkage="ward"

)

hierarchical_labels = final_hierarchical.fit_predict(X_scaled_df)

# Add labels to dataset

df_features["Hierarchical_Cluster"] = hierarchical_labels

# Cluster sizes

print("Hierarchical cluster sizes:")

display(

    df_features["Hierarchical_Cluster"]

    .value_counts()

    .sort_index()

    .to_frame("Customers")

)
```

Hierarchical_Cluster	
0	178
1	262

The final hierarchical clustering model produced two reasonably balanced customer groups:

- **Cluster 0:** 178 customers
- **Cluster 1:** 262 customers

This indicates that the model did not create an extremely small or dominant cluster.

PCA Visualisation

PCA visualisation

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(
    pca_df["PC1"],
    pca_df["PC2"],
    c=hierarchical_labels,
    alpha=0.7
)
```

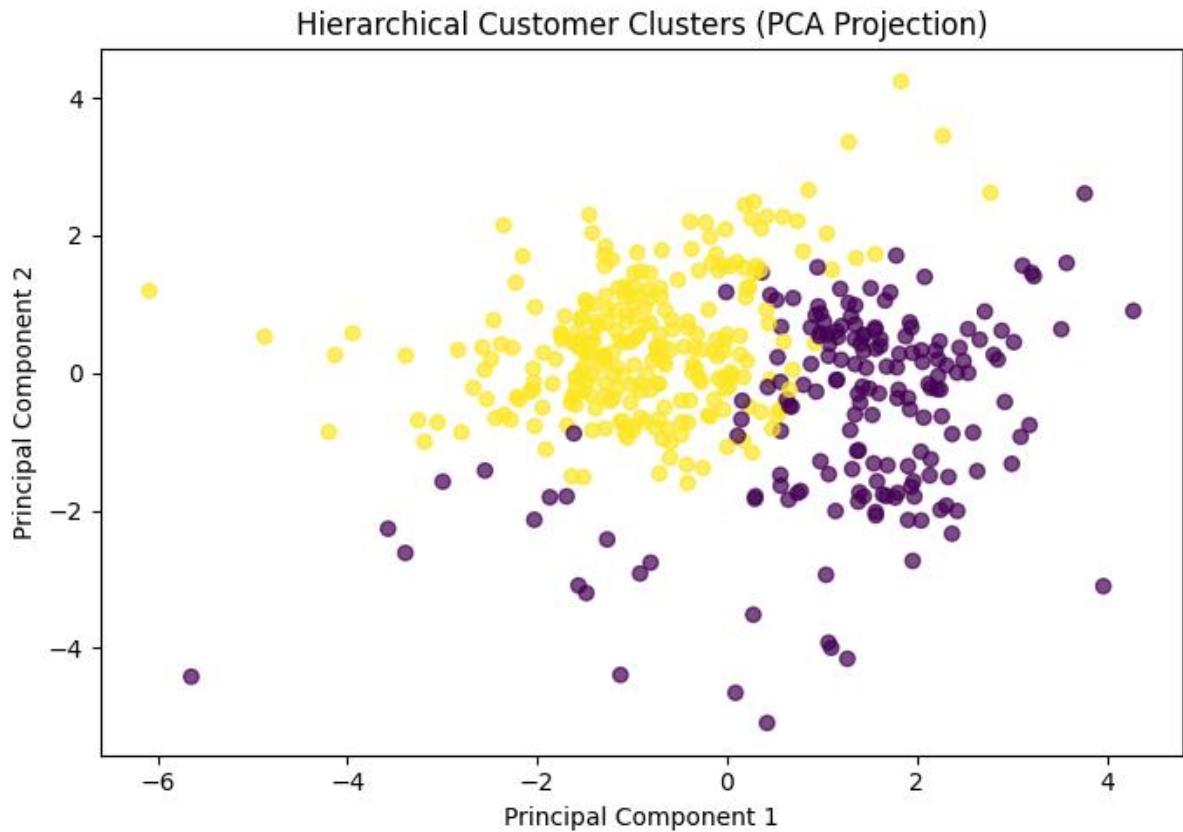
```
plt.xlabel("Principal Component 1")
```

```
plt.ylabel("Principal Component 2")
```

```
plt.title("Hierarchical Customer Clusters (PCA Projection)")
```

```
plt.tight_layout()
```

```
plt.show()
```



The PCA plot shows that the two hierarchical clusters are mainly separated along **Principal Component 1**.

Cluster 0 is mostly located on the right side of the plot, while Cluster 1 is mainly located on the left. Some overlap remains near the centre, which is consistent with the moderate Silhouette Score of **0.258**.

Median spending profile

Median spending profile by cluster:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
Hierarchical_Cluster						
0	4499.5	7339.0	11099.0	957.0	4584.0	1189.0
1	11113.0	2028.0	2583.5	2382.0	358.5	864.0

The median spending profiles show two different customer groups:

- **Cluster 0:** Higher spending on **Milk, Grocery, Detergents_Paper** and **Delicassen**, but lower spending on Fresh and Frozen products.

- **Cluster 1:** Higher spending on **Fresh and Frozen products**, but lower spending across the other categories.

These customer groups are similar to those identified by K-Means, although the cluster numbers are reversed.

DBSCAN Clustering

DBSCAN groups customers located closely together and labels customers in low-density areas as noise or outliers. Unlike K-Means and hierarchical clustering, it does not require the number of clusters to be selected beforehand.

K-Distance Plot

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.neighbors import NearestNeighbors

# Initial DBSCAN min_samples value
min_samples = 12

# Calculate distances to the 12th nearest neighbour
nearest_neighbors = NearestNeighbors(
    n_neighbors=min_samples
)

nearest_neighbors.fit(X_scaled_df)

distances, indices = nearest_neighbors.kneighbors(X_scaled_df)

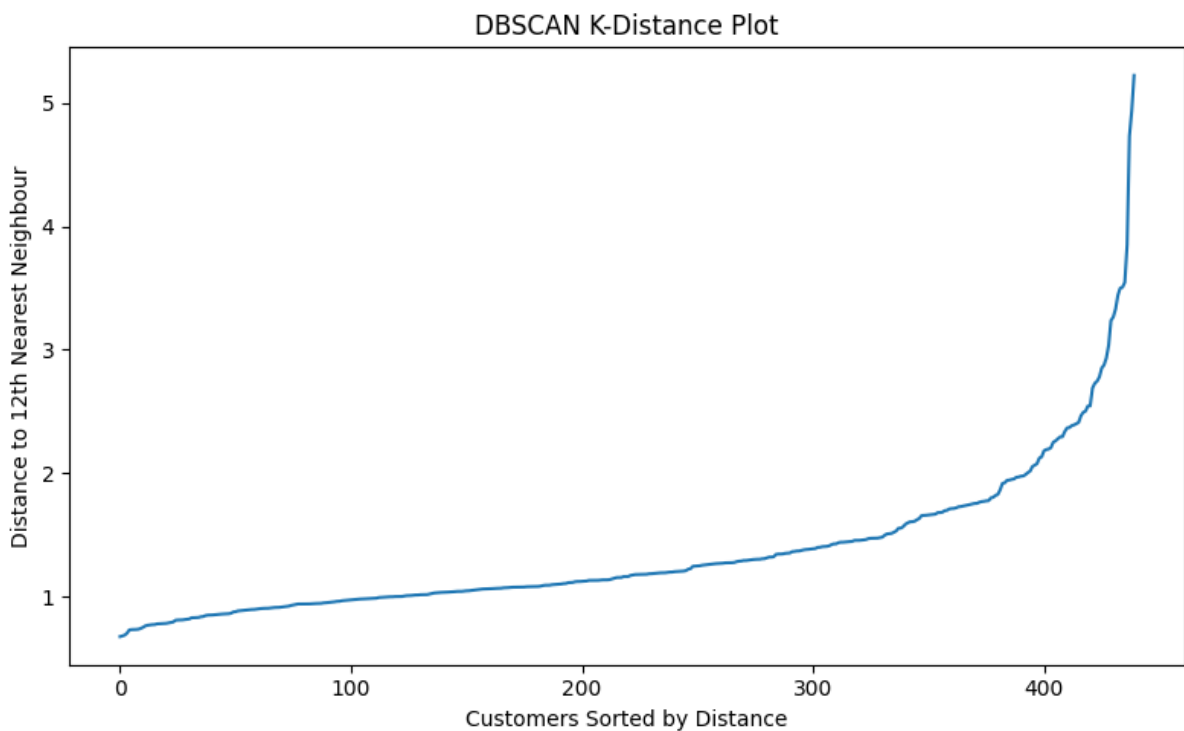
# Sort the distance to the furthest neighbour
k_distances = np.sort(distances[:, -1])
```

```
# Plot the K-distance graph
plt.figure(figsize=(8, 5))

plt.plot(k_distances)

plt.xlabel("Customers Sorted by Distance")
plt.ylabel(f"Distance to {min_samples}th Nearest Neighbour")
plt.title("DBSCAN K-Distance Plot")

plt.tight_layout()
plt.show()
```



The curve begins to rise sharply at a distance of approximately **2.0 to 2.4**. This suggests that suitable **eps** values are likely within this range.

eps is the maximum distance used to decide whether customers are close enough to belong to the same neighbourhood.

- A **small eps** creates tighter clusters and may label many customers as noise.
- A **large eps** connects more customers together and may merge separate groups into one large cluster.

DBSCAN Parameters

```
from sklearn.cluster import DBSCAN

from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score
)

dbscan_results = []

eps_values = [1.6, 1.8, 2.0, 2.2, 2.4, 2.6]
min_samples_values = [6, 9, 12, 15]

for eps in eps_values:
    for min_samples in min_samples_values:

        model = DBSCAN(
            eps=eps,
            min_samples=min_samples
        )

        labels = model.fit_predict(X_scaled_df)

        cluster_labels = set(labels)
```

```
number_of_clusters = len(cluster_labels - {-1})
noise_points = list(labels).count(-1)

# Evaluate only when at least two clusters exist
valid_mask = labels != -1
valid_labels = labels[valid_mask]
valid_data = X_scaled_df[valid_mask]

if len(set(valid_labels)) >= 2:
    silhouette = silhouette_score(
        valid_data,
        valid_labels
    )
    davies_bouldin = davies_bouldin_score(
        valid_data,
        valid_labels
    )
    calinski_harabasz = calinski_harabasz_score(
        valid_data,
        valid_labels
    )
else:
    silhouette = None
    davies_bouldin = None
    calinski_harabasz = None

dbscan_results.append({
    "eps": eps,
```



```
"min_samples": min_samples,  
"Clusters": number_of_clusters,  
"Noise Points": noise_points,  
"Silhouette Score": silhouette,  
"Davies-Bouldin Index": davies_bouldin,  
"Calinski-Harabasz Score": calinski_harabasz  
})
```

```
dbscan_results_df = pd.DataFrame(dbscan_results)
```

```
display(dbscan_results_df.round(3))
```

	eps	min_samples	Clusters	Noise Points	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
0	1.6	6	1	37	None	None	None
1	1.6	9	1	41	None	None	None
2	1.6	12	1	42	None	None	None
3	1.6	15	1	43	None	None	None
4	1.8	6	1	22	None	None	None
5	1.8	9	1	25	None	None	None
6	1.8	12	1	27	None	None	None
7	1.8	15	1	30	None	None	None
8	2.0	6	1	15	None	None	None
9	2.0	9	1	16	None	None	None
10	2.0	12	1	17	None	None	None
11	2.0	15	1	18	None	None	None
12	2.2	6	1	8	None	None	None
13	2.2	9	1	10	None	None	None
14	2.2	12	1	11	None	None	None
15	2.2	15	1	12	None	None	None
16	2.4	6	1	5	None	None	None
17	2.4	9	1	7	None	None	None
18	2.4	12	1	8	None	None	None
19	2.4	15	1	8	None	None	None
20	2.6	6	1	4	None	None	None
21	2.6	9	1	5	None	None	None
22	2.6	12	1	5	None	None	None
23	2.6	15	1	6	None	None	None

All tested combinations produced only **one cluster**, with some customers classified as noise. As eps increased, the number of noise points decreased from 37 to 4 because more customers were connected into the same cluster.

The evaluation metrics were shown as None because Silhouette, Davies–Bouldin and Calinski–Harabasz scores require at least **two clusters**. This suggests that the tested eps range of **1.6–2.6 was too large** and merged nearly all customers together.

We will now test smaller eps values:

```
eps_values = [0.5, 0.7, 0.9, 1.1, 1.3, 1.5]
```

```
min_samples_values = [4, 6, 8, 10, 12]
```

```
dbscan_results = []
```

```
for eps in eps_values:
    for min_samples in min_samples_values:

        model = DBSCAN(
            eps=eps,
            min_samples=min_samples
        )

        labels = model.fit_predict(X_scaled_df)

        number_of_clusters = len(set(labels) - {-1})
        noise_points = sum(labels == -1)

        valid_mask = labels != -1
        valid_labels = labels[valid_mask]
        valid_data = X_scaled_df.loc[valid_mask]

        if number_of_clusters >= 2:
            silhouette = silhouette_score(valid_data, valid_labels)
            davies_bouldin = davies_bouldin_score(
                valid_data,
                valid_labels
            )
            calinski_harabasz = calinski_harabasz_score(
                valid_data,
                valid_labels
            )
```

else:

 silhouette = None

 davies_bouldin = None

 calinski_harabasz = None

dbscan_results.append({

 "eps": eps,

 "min_samples": min_samples,

 "Clusters": number_of_clusters,

 "Noise Points": noise_points,

 "Silhouette Score": silhouette,

 "Davies-Bouldin Index": davies_bouldin,

 "Calinski-Harabasz Score": calinski_harabasz

})

dbscan_results_df = pd.DataFrame(dbscan_results)

display(dbscan_results_df.round(3))

index	eps	min_samples	Clusters	Noise Points	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
0	0.5	4	6	415	0.439	0.73	63.618
1	0.5	6	0	440	NaN	NaN	NaN
2	0.5	8	0	440	NaN	NaN	NaN
3	0.5	10	0	440	NaN	NaN	NaN
4	0.5	12	0	440	NaN	NaN	NaN
5	0.7	4	13	292	0.106	1.021	38.748
6	0.7	6	5	365	0.245	1.154	55.666
7	0.7	8	3	397	0.553	0.695	93.862
8	0.7	10	1	422	NaN	NaN	NaN
9	0.7	12	1	423	NaN	NaN	NaN
10	0.9	4	4	151	-0.136	1.296	19.583
11	0.9	6	4	175	0.024	0.96	13.595
12	0.9	8	1	211	NaN	NaN	NaN
13	0.9	10	2	248	0.461	0.858	236.714
14	0.9	12	2	273	0.486	0.807	232.542
15	1.1	4	2	82	0.154	0.956	5.121
16	1.1	6	1	102	NaN	NaN	NaN
17	1.1	8	1	110	NaN	NaN	NaN
18	1.1	10	2	122	0.261	0.785	19.364
19	1.1	12	1	140	NaN	NaN	NaN
20	1.3	4	1	53	NaN	NaN	NaN
21	1.3	6	1	64	NaN	NaN	NaN
22	1.3	8	1	69	NaN	NaN	NaN
23	1.3	10	1	72	NaN	NaN	NaN
24	1.3	12	1	79	NaN	NaN	NaN
25	1.5	4	1	40	NaN	NaN	NaN
26	1.5	6	1	41	NaN	NaN	NaN
27	1.5	8	1	45	NaN	NaN	NaN
28	1.5	10	1	48	NaN	NaN	NaN
29	1.5	12	1	51	NaN	NaN	NaN

The highest Silhouette Score was **0.553** at eps = 0.7 and min_samples = 8. However, this model classified **397 of 440 customers as noise**, leaving only 43 customers in clusters. Therefore, the score is misleading and the model is not useful for customer segmentation.

Similarly, the models using eps = 0.9 produced two or more clusters but still classified more than half of the customers as noise.

The most reasonable balance appears around **eps = 1.1**, where fewer customers are excluded. However, the evaluation scores are weaker, and some parameter combinations still produce only one cluster.

Overall, these results suggest that DBSCAN may not be well suited to this dataset because it either:

- Creates one large cluster, or
- Produces multiple clusters while treating too many customers as noise.

Before making the final conclusion, test a narrower range around eps = 0.9–1.1:

Model Comparison and Final Selection

The three clustering methods produced different results:

- **K-Means** achieved the best overall performance, with a Silhouette Score of **0.290**, a Davies–Bouldin Index of **1.352**, and a Calinski–Harabasz Score of **189.050**. It created two reasonably balanced clusters containing **252 and 188 customers**.
- **Hierarchical Clustering** also identified two similar customer groups but performed slightly worse, with a Silhouette Score of **0.258**, a Davies–Bouldin Index of **1.600**, and a Calinski–Harabasz Score of **134.624**.
- **DBSCAN** was unsuitable because it either created one large cluster or classified a large proportion of customers as noise. Its higher Silhouette Scores were based on only a small number of non-noise customers and were therefore not directly comparable with the other models.

Final Model Selection

K-Means with two clusters was selected as the final model because it:

- Achieved the strongest overall evaluation scores.
- Produced reasonably balanced customer groups.
- Created clear and understandable spending profiles.
- Assigned every customer to a cluster.
- Can assign future customers to an existing segment using the trained cluster centres.

The final segments were:

- **Fresh and Frozen-focused customers**
- **Grocery, Milk and Detergents-focused customers**

These segments can support targeted promotions, cross-selling, customer prioritisation and account-management strategies.

Saving the Final Model

The K-Means model, scaler, PCA model, selected feature names and cluster profiles has been saved for reproducibility.

```
import os

import json

import joblib

import pandas as pd


from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA


# Create model directory

os.makedirs("models", exist_ok=True)


# Refit preprocessing and final model

X = df[selected_features].copy()

X_log = np.log1p(X)


final_scaler = StandardScaler()

X_scaled = final_scaler.fit_transform(X_log)


final_kmeans = KMeans(

    n_clusters=2,

    random_state=42,

    n_init=10

)


final_labels = final_kmeans.fit_predict(X_scaled)


# Fit PCA for visualisation
```

```
final_pca = PCA(n_components=2)

final_pca.fit(X_scaled)


# Save model objects

joblib.dump(final_kmeans, "models/kmeans_model.joblib")
joblib.dump(final_scaler, "models/standard_scaler.joblib")
joblib.dump(final_pca, "models/pca_model.joblib")


# Save selected features

with open("models/selected_features.json", "w") as file:
    json.dump(selected_features, file, indent=4)


# Save cluster profiles

df_model_results = df.copy()
df_model_results["KMeans_Cluster"] = final_labels


cluster_profiles = (
    df_model_results
    .groupby("KMeans_Cluster")[selected_features]
    .median()
)

cluster_profiles.to_csv("models/kmeans_cluster_profiles.csv")

print("Final model and preprocessing objects saved successfully.")
```


Step 5: Critical Thinking, Ethical AI & Bias Auditing

Model Explainability Using SHAP

K-Means does not directly provide feature-level explanations. Therefore, a **Random Forest surrogate model** will be trained to reproduce the final K-Means cluster assignments. **SHAP** (SHapley Additive exPlanations) will then explain which spending categories influence assignment towards Cluster 0 or Cluster 1. The surrogate model's accuracy must first be checked to confirm that it closely represents the original clustering model. SHAP's TreeExplainer is designed to explain tree-based ensemble models. The definitions of Random Forest surrogate model and SHAP is explained below,

- **Random Forest surrogate model:** A supervised Random Forest classifier trained to imitate the K-Means cluster assignments so the factors influencing each cluster can be explained.
- **SHAP (SHapley Additive exPlanations):** An explainability method that measures how much each feature contributes to pushing a customer towards a particular cluster.

```
# Install SHAP if required
```

```
!pip install shap -q
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import shap
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
# -----  
# 1. Recreate the final K-Means cluster assignments  
# -----  
  
selected_features = [  
    "Fresh",  
    "Milk",  
    "Grocery",  
    "Frozen",  
    "Detergents_Paper",  
    "Delicassen"  
]  
  
X_original = df[selected_features].copy()  
  
X_log = np.log1p(X_original)  
  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_log)  
  
final_kmeans = KMeans(  
    n_clusters=2,  
    random_state=42,  
    n_init=10  
)  
  
cluster_labels = final_kmeans.fit_predict(X_scaled)
```

```

# -----
# 2. Train a surrogate Random Forest model
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X_original,
    cluster_labels,
    test_size=0.20,
    random_state=42,
    stratify=cluster_labels
)

surrogate_model = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    class_weight="balanced"
)

surrogate_model.fit(X_train, y_train)

predictions = surrogate_model.predict(X_test)

print("Surrogate model accuracy:")
print(round(accuracy_score(y_test, predictions), 3))

print("\nClassification report:")
print(classification_report(y_test, predictions))

```

```
print("Confusion matrix:")
display(pd.DataFrame(
    confusion_matrix(y_test, predictions),
    index=["Actual Cluster 0", "Actual Cluster 1"],
    columns=["Predicted Cluster 0", "Predicted Cluster 1"]
))
```

```
# -----
```

```
# 3. Calculate SHAP values
```

```
# -----
```

```
X_test_reset = X_test.reset_index(drop=True)
y_test_reset = pd.Series(y_test).reset_index(drop=True)
```

```
explainer = shap.Explainer(
    surrogate_model,
    X_train
)
```

```
shap_output = explainer(X_test_reset)
```

```
# Select SHAP values explaining Cluster 1
```

```
if shap_output.values.ndim == 3:
```

```
    shap_cluster_1 = shap_output[:, :, 1]
```

```
else:
```

```
    shap_cluster_1 = shap_output
```

```
# -----
```

4. Global feature importance

```
shap.plots.bar(  
    shap_cluster_1,  
    max_display=6  
)
```

5. SHAP summary plot

```
shap.plots.beeswarm(  
    shap_cluster_1,  
    max_display=6  
)
```

6. Mean absolute SHAP importance table

```
shap_importance = pd.DataFrame({  
    "Feature": selected_features,  
    "Mean Absolute SHAP Value":  
        np.abs(shap_cluster_1.values).mean(axis=0)  
}).sort_values(  
    "Mean Absolute SHAP Value",  
    ascending=False
```

```
)
```

```
print("\nSHAP feature importance:")
```

```
display(shap_importance.round(4))
```

```
# -----
```

```
# 7. Individual customer explanations
```

```
# -----
```

```
cluster_0_position = np.where(y_test_reset == 0)[0][0]
```

```
cluster_1_position = np.where(y_test_reset == 1)[0][0]
```

```
print("Example customer assigned to Cluster 0:")
```

```
shap.plots.waterfall(
```

```
    shap_cluster_1[cluster_0_position],
```

```
    max_display=6
```

```
)
```

```
print("Example customer assigned to Cluster 1:")
```

```
shap.plots.waterfall(
```

```
    shap_cluster_1[cluster_1_position],
```

```
    max_display=6
```

```
)
```

Surrogate model accuracy:
1.0

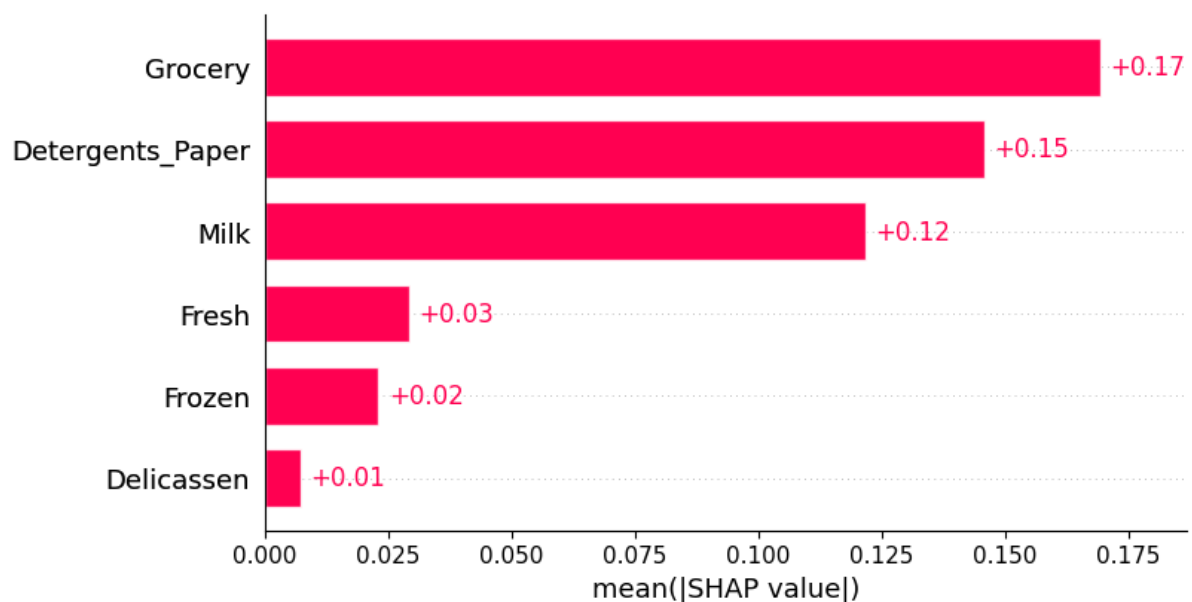
Classification report:

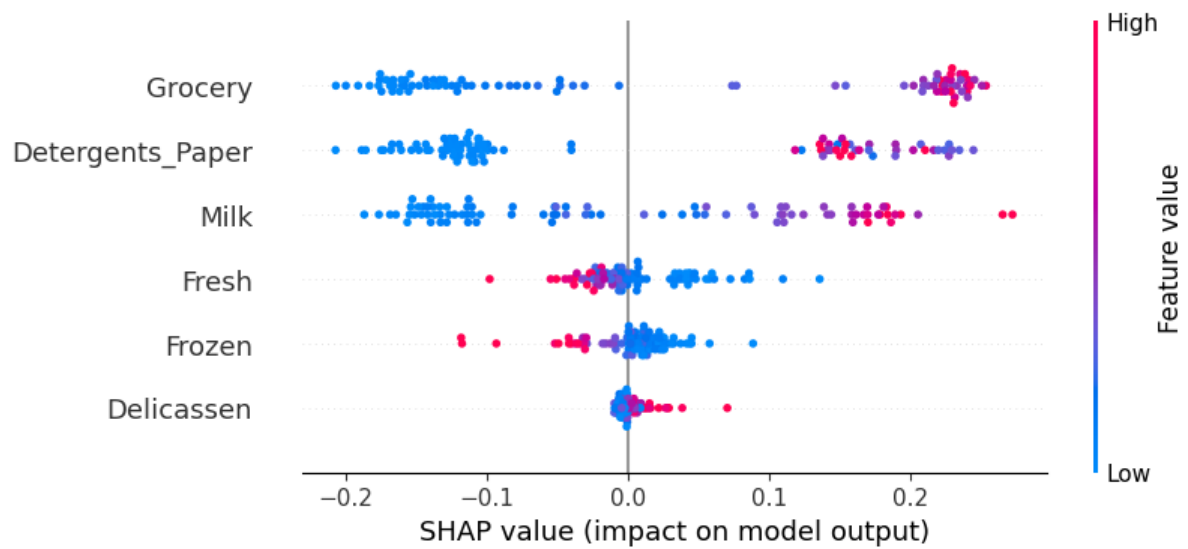
	precision	recall	f1-score	support
0	1.00	1.00	1.00	50
1	1.00	1.00	1.00	38
accuracy			1.00	88
macro avg	1.00	1.00	1.00	88
weighted avg	1.00	1.00	1.00	88

Confusion matrix:

	Predicted Cluster 0	Predicted Cluster 1
Actual Cluster 0	50	0
Actual Cluster 1	0	38

WARNING:shap:Background dataset has 352 samples but max_samples=100.
Subsampling to 100 samples for SHAP value computation. To use all samples, set
max_samples=352 when initializing the masker.

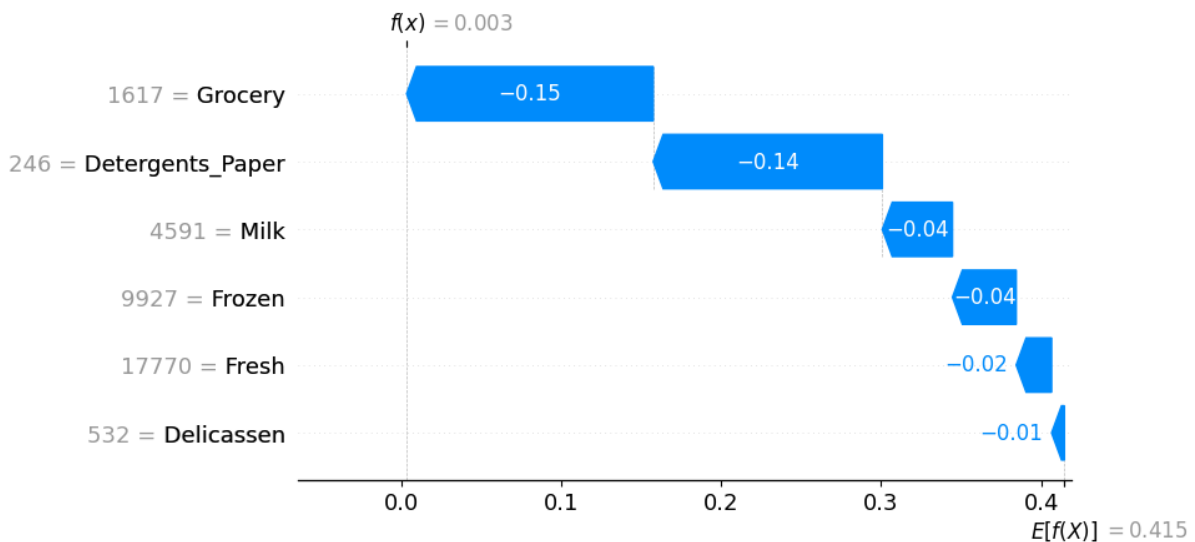




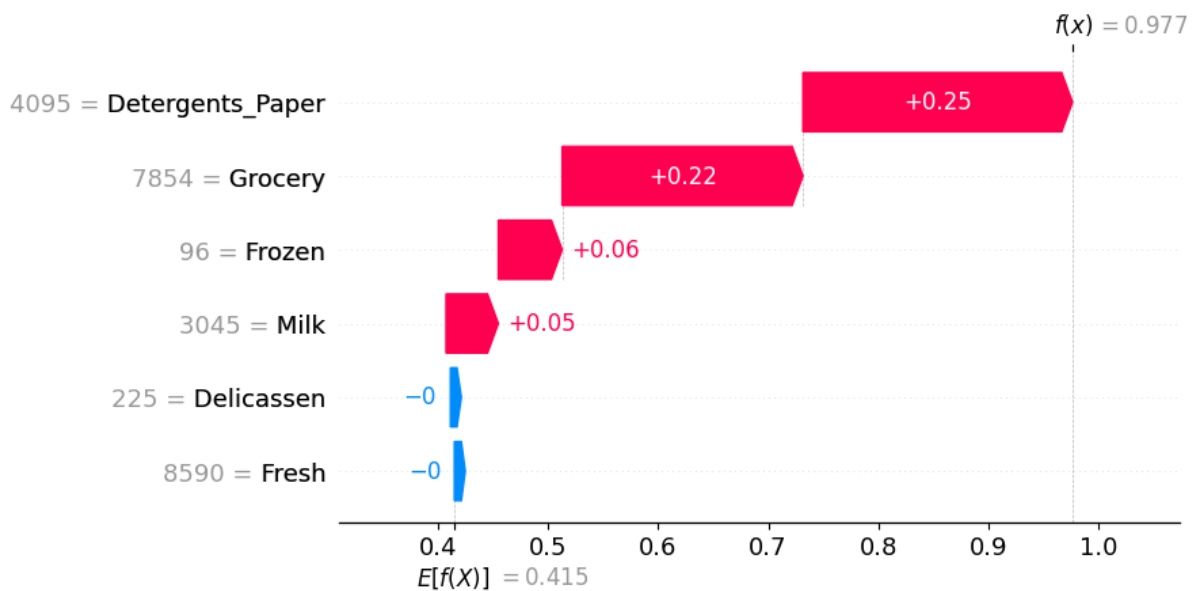
SHAP feature importance:

	Feature	Mean Absolute SHAP Value
2	Grocery	0.1694
4	Detergents_Paper	0.1458
1	Milk	0.1218
0	Fresh	0.0292
3	Frozen	0.0230
5	Delicassen	0.0072

Example customer assigned to Cluster 0:



Example customer assigned to Cluster 1:



The Random Forest surrogate model achieved **100% accuracy** on the test data, correctly reproducing all 88 K-Means cluster assignments. This indicates that it provides a reliable approximation of the clustering model for SHAP analysis.

The SHAP results showed that **Grocery, Detergents_Paper and Milk** were the most influential features. Higher spending in these categories pushed customers towards **Cluster 1**, while lower spending pushed them towards Cluster 0.

Higher **Fresh and Frozen** spending generally pushed customers towards **Cluster 0**, which is consistent with the earlier cluster profiles. **Delicassen** had the smallest influence on cluster assignment.

Overall, the SHAP analysis confirms that:

- **Cluster 0** represents customers focused on Fresh and Frozen products.

- **Cluster 1** represents customers focused on Grocery, Milk and Detergents_Paper.

The warning shown above the SHAP plot only indicates that SHAP sampled 100 background records to reduce computation time; it does not indicate an error.

Model Limitations

Data Limitations

- The dataset contains only **440 customers from one wholesale distributor**.
- No missing values were found, but several useful business variables are absent, including profit margin, customer size, transaction dates, retention and campaign response.
- The data contains only annual spending totals, so seasonal and monthly purchasing patterns cannot be examined.
- The monetary unit is not clearly defined, limiting detailed financial interpretation.
- Channel and Region are imbalanced, with most customers belonging to Channel 1 and Region 3.
- Extreme spending values may still influence the analysis, although log transformation reduced their effect.

Model Limitations

- K-Means assumes compact clusters and may not fully represent irregular customer groups.
- Every customer is forced into a cluster, even when their spending pattern falls between both segments.
- The Silhouette Score of **0.290** indicates some overlap between the clusters.
- The two-cluster model may simplify customer behaviour and hide smaller customer segments.
- Results may change depending on preprocessing, feature selection and the number of clusters selected.
- SHAP explains the surrogate Random Forest model, which imitates the K-Means assignments rather than directly explaining K-Means.

Generalisability Limitations

- The clusters may not apply to other wholesalers, industries or customer populations.
- The dataset may represent a particular historical period and may not reflect current customer behaviour.
- Customer spending patterns may differ across countries, regions and economic conditions.

- The model should be validated using newer company data before being used for major business decisions.
- A business pilot should be conducted to measure whether the segments improve sales, cross-selling or customer engagement.

Summary

Overall, the model provides a practical basis for customer segmentation and can support targeted sales, cross-selling and account-management strategies. However, the segments should first be validated through a business pilot and reviewed alongside customer knowledge before being used for major commercial decisions.

Bias and Fairness Audit

The dataset does not contain protected attributes such as gender, race, age or socioeconomic status. Therefore, **Channel and Region** will be used as the available grouping variables. These are business attributes rather than protected characteristics, so the analysis should be described as an **operational fairness and representation audit**.

Because Cluster 0 and Cluster 1 are not automatically favourable or unfavourable outcomes, equalised odds is not applicable. Instead, the audit will examine:

- Representation of each Channel and Region within the clusters
- Differences in cluster-assignment rates between groups
- Representation parity ratios
- Statistical association using the Chi-square test and Cramér's V

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy.stats import chi2_contingency
```

```
# Ensure cluster labels are present in the current 'df' session
```

```
# 'cluster_labels' and 'hierarchical_labels' are stored in the kernel
```

```
df["KMeans_Cluster"] = cluster_labels
```

```
df["Hierarchical_Cluster"] = hierarchical_labels
```

```

# Prepare fairness audit dataset
audit_df = df[["Channel", "Region", "KMeans_Cluster"]].copy()

# Mapping for readability
audit_df["Channel"] = audit_df["Channel"].map({1: "Channel 1", 2: "Channel 2"})
audit_df["Region"] = audit_df["Region"].map({1: "Region 1", 2: "Region 2", 3: "Region 3"})

# 1. Representation counts
for attribute in ["Channel", "Region"]:
    print(f"\nCustomer counts by {attribute} and cluster:")
    display(pd.crosstab(audit_df[attribute], audit_df["KMeans_Cluster"]))

# 2. Cluster assignment rates within each group
for attribute in ["Channel", "Region"]:
    print(f"\nCluster assignment percentages by {attribute}:")
    display((pd.crosstab(audit_df[attribute], audit_df["KMeans_Cluster"],
normalize="index") * 100).round(2))

# 3. Composition of each cluster
for attribute in ["Channel", "Region"]:
    print(f"\nComposition of each cluster by {attribute}:")
    display((pd.crosstab(audit_df[attribute], audit_df["KMeans_Cluster"],
normalize="columns") * 100).round(2))

# 4. Representation parity metrics
parity_results = []

for attribute in ["Channel", "Region"]:

```

```

rates = pd.crosstab(audit_df[attribute], audit_df["KMeans_Cluster"],
normalize="index")

for cluster in rates.columns:

    c_rates = rates[cluster]

    parity_results.append({

        "Attribute": attribute,

        "Cluster": cluster,

        "Max Rate": c_rates.max(),

        "Min Rate": c_rates.min(),

        "Parity Difference": c_rates.max() - c_rates.min(),

        "Parity Ratio": c_rates.min() / c_rates.max() if c_rates.max() > 0 else np.nan

    })

print("\nRepresentation parity results:")

display(pd.DataFrame(parity_results).round(3))

```

5. Chi-square and Cramer's V

```

association_results = []

for attribute in ["Channel", "Region"]:

    table = pd.crosstab(audit_df[attribute], audit_df["KMeans_Cluster"])

    chi2, p_val, dof, ex = chi2_contingency(table)

    n = table.sum().sum()

    r, c = table.shape

    # Cramer's V calculation

    v = np.sqrt((chi2 / n) / min(r - 1, c - 1)) if n > 0 else 0

    association_results.append({

        "Attribute": attribute,

        "Chi-Square": chi2,

```

```

    "P-Value": p_val,
    "Cramers V": v
})

```

```

print("\nAssociation between groups and cluster membership:")
display(pd.DataFrame(association_results).round(4))

```

Customer counts by Channel and cluster:

KMeans_Cluster	0	1
Channel		
Channel 1	244	54
Channel 2	8	134

Customer counts by Region and cluster:

KMeans_Cluster	0	1
Region		
Region 1	48	29
Region 2	28	19
Region 3	176	140

Cluster assignment percentages by Channel:

KMeans_Cluster	0	1
Channel		
Channel 1	81.88	18.12
Channel 2	5.63	94.37

Cluster assignment percentages by Region:

KMeans_Cluster	0	1
Region		
Region 1	62.34	37.66
Region 2	59.57	40.43
Region 3	55.70	44.30

Composition of each cluster by Channel:

KMeans_Cluster	0	1
Channel		
Channel 1	96.83	28.72
Channel 2	3.17	71.28

Composition of each cluster by Region:

KMeans_Cluster	0	1
Region		
Region 1	19.05	15.43
Region 2	11.11	10.11
Region 3	69.84	74.47

Representation parity results:

	Attribute	Cluster	Max Rate	Min Rate	Parity Difference	Parity Ratio
0	Channel	0	0.819	0.056	0.762	0.069
1	Channel	1	0.944	0.181	0.762	0.192
2	Region	0	0.623	0.557	0.066	0.893
3	Region	1	0.443	0.377	0.066	0.850

Association between groups and cluster membership:

	Attribute	Chi-Square	P-Value	Cramers V
0	Channel	225.3633	0.0000	0.7157
1	Region	1.2299	0.5407	0.0529

Channel Analysis

A strong relationship was found between Channel and cluster membership:

- **81.88% of Channel 1 customers** were assigned to Cluster 0.

- **94.37% of Channel 2 customers** were assigned to Cluster 1.
- The parity differences were **0.762**, while the parity ratios were low at **0.069 and 0.192**, showing a large difference in assignment rates.
- The Chi-square test was statistically significant, with **$p < 0.001$** .
- Cramér's V was **0.716**, indicating a strong association between Channel and cluster membership.

This does not automatically indicate unfairness because Channel is a business characteristic and may genuinely reflect different purchasing behaviours. However, the business should avoid using the clusters in a way that unfairly reduces service or opportunities for one channel.

Region Analysis

Cluster assignment was much more balanced across regions:

- Cluster 0 assignment ranged from **55.70% to 62.34%**.
- Cluster 1 assignment ranged from **37.66% to 44.30%**.
- The parity difference was only **0.066**, and parity ratios were **0.893 and 0.850**, which are relatively close to 1.
- The Chi-square test produced **$p = 0.5407$** .
- Cramér's V was **0.0529**, indicating a very weak association.

Therefore, there is no meaningful evidence that Region strongly influenced cluster assignment.

Overall Finding

The audit found a strong association between **Channel and customer segment**, but little evidence of **regional imbalance**. Since the dataset does not include protected attributes such as gender, age or race, the audit is limited to operational fairness rather than a full demographic fairness assessment.

Mitigation Strategies

Data Improvements

- Collect additional variables such as customer size, profit margin, order frequency, retention and campaign response.
- Use more recent and multi-year data to capture changes in customer behaviour.

- Review whether Channel and Region groups are adequately represented.
- Include protected demographic attributes only where legally and ethically appropriate for future fairness testing.

Model and Validation Improvements

- Reassess the clusters regularly because customer spending patterns may change.
- Compare results across different preprocessing methods, random seeds and customer samples.
- Review customers near the cluster boundaries rather than automatically applying a segment strategy.
- Test whether more detailed sub-segments provide additional business value without creating unstable groups.

Fairness Mitigation

- Do not use cluster membership to deny customers service, pricing access or account support.
- Investigate the strong Channel association before making commercial decisions, as it may reflect genuine purchasing differences rather than unfair bias.
- Ensure that smaller customer groups are not ignored simply because they have lower spending.
- Monitor whether campaigns based on the clusters produce consistently worse outcomes for a particular Channel or Region.

Monitoring and Governance

- Require human review before major decisions are made using cluster assignments.
- Document the model assumptions, preprocessing steps and limitations.
- Track sales uplift, customer engagement and retention for each segment.
- Retrain and re-audit the model when new data becomes available.

These strategies allow the business to use the customer segments while reducing the risk of unfair or misleading decisions.

Ethical AI Summary and Recommendations

The customer segmentation model provides useful business insights by identifying two main purchasing groups. However, the model should be used as a decision-support tool rather than an automatic decision-making system.

Key ethical considerations include:

- Customers should not receive reduced service, higher prices or fewer opportunities solely because of their cluster.
- The strong relationship between Channel and cluster membership should be reviewed before applying different strategies.
- Smaller or lower-spending customer groups should not be ignored.
- Cluster assignments should be reviewed regularly as customer behaviour changes.
- Human judgement should remain involved in important sales and account-management decisions.
- The model, assumptions, limitations and preprocessing steps should be clearly documented.

Final Recommendations

- Use the clusters to support targeted promotions, cross-selling and account-management strategies.
- Conduct a small business pilot before applying the segmentation widely.
- Measure outcomes such as sales growth, engagement, retention and campaign response.
- Review customers close to cluster boundaries individually.
- Update and re-audit the model when new sales data becomes available.
- Continue monitoring Channel and Region outcomes to identify possible unfair impacts.

Overall, the model can provide practical business value when used transparently, monitored regularly and combined with human judgement